

A Weakly Hybrid Decoding Algorithm for Staircase Codes via Multi-Level Bit Marking

Yi Lei^{ID}, *Member, IEEE*, Xiang Wang, Bin Chen^{ID}, *Senior Member, IEEE*, Minghua Cao^{ID}, and Zhongyi Guo^{ID}

Abstract—Reducing the complexity of soft-decision (SD) decoding algorithm or improving the performance of hard-decision (HD) decoding algorithm becomes an emerging trend on forward error corrections to ensure reliable optical transmission with high performance and low cost. In this paper, a hybrid decoding algorithm is proposed for staircase codes (SCCs), which performs HD decoding with the help of channel soft information via multi-level bit-marking (MLBM). The HD and SD in the proposed MLBM decoding algorithm is *weakly hybrid* that the soft information is only used for one-shot bit marking and does not involve in the decoding directly, thus providing good performance-complexity tradeoff. The marked bits indicate four reliability levels that are used to prevent miscorrections by checking conflicts with highly reliable bits and to enhance the error-correcting capability of the HD decoder by flipping unreliable bits. Particularly, three priority levels are set for bit flipping by combining with the decoding statuses of the associated component codes. The decoding results will, in turn, help to update the marked bits among the four reliability levels. Numerical results show up to 1.22 dB gains with steeper waterfall region and lower error floor when compared to conventional HD decoding for SCCs, at the cost of approximately a two-fold increase in the memory storage.

Index Terms—Optical fiber communications, forward error correction, staircase codes, hard-decision decoding, soft-decision decoding.

I. INTRODUCTION

DRIVEN by the increasing demand for high-speed data transmission, the standardization of 400G is completed or nearly completed, and 800G techniques become the new focus for next generation optical transport networks (OTNs) [1], where forward error correction (FEC) technology is of key importance [2], [3], [4]. With the continuous increase in target rates, advanced FEC schemes that balance complexity and performance to provide higher coding gain, lower latency, and

lower power consumption are required. Therefore, reducing the complexity of soft-decision (SD) decoding algorithm and improving the performance of hard-decision (HD) decoding algorithm are two emerging and important trends on FEC to ensure reliable optical transmission with high performance and considerable cost [5], [6], [7].

Regarding to the HD-FECs, the product-like codes, e.g., product codes (PCs) [8], staircase codes (SCCs) [9] and zipper codes [10], have attained much attention in optical communication fields, featuring two parity constraints for each information bit. These codes are typically constructed from short component codes, e.g., Bose-Chaudhuri-Hocquenghem (BCH) codes, and are decoded with iterative bounded distance decoding (BDD), yielding superior net coding gain. Particularly, SCC has been incorporated into industry standards, such as ITU-T G.709.2/Y.1331.2 [11] and 400G ZR [12].

To pursue higher coding gain of the product-like codes, a straightforward way is to use high-performance SD-FEC codes as component codes, such as polar codes [13], [14], low-density parity-check (LDPC) codes [15], and etc. Unsurprisingly, the achieved gains are significant, but at the cost of greatly increased decoding complexity and power consumption. Although several works have put effort on reducing the complexity of LDPC codes and polar codes [16], [17], [18], SD decoding is not the best candidates for product-like codes to meet the performance and power consumption demand for high data rate applications, or at least, not yet.

To keep the advantage in complexity of product-like codes, there are works that still focus on HD decoding itself. To lower error floor, the work in [19] proposes to locate stall patterns using the intersection of nonzero-syndrome component codes, and resolve the stall pattern through bit flipping. To mitigate the influence of BDD miscorrection, an anchor-based decoding (AD) algorithm is proposed in [20] by tracking the conflicts between the row and column component codes. Further, an enhanced AD algorithm is proposed in [21], which uses hard reliability scores obtained by the decoding results of BDD. To improve the energy-efficiency (i.e., to enable to use 1-error-correcting capability component codes while still achieving error floor below 10^{-15}), a generalized SCC is proposed recently, where each bit is protected by an arbitrary number of component codewords rather than two [22]. Although these methods are simple for hardware implementation, the performance is limited, still with significant gaps to the HD Shannon limit (equivalent to binary symmetric channel) as well as the SD-FEC codes.

Received 24 May 2025; revised 9 October 2025; accepted 8 December 2025. Date of publication 23 December 2025; date of current version 2 January 2026. This work is supported by the Fundamental Research Funds for the Central Universities under Grant JZ2024HGTG0312, the NSFC Program (No. 62001151, 62171175, 62265010), and the Anhui Provincial Natural Science Foundation (No. 2508085J041). The associate editor coordinating the review of this article and approving it for publication was M. Shirvanimoghaddam. (*Corresponding author: Bin Chen.*)

Yi Lei, Bin Chen, and Zhongyi Guo are with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230009, China (e-mail: leiyi@hfut.edu.cn; bin.chen@hfut.edu.cn; guozhongyi@hfut.edu.cn).

Xiang Wang was with the School of Computer Science and Information Engineering, Hefei University of Technology, Hefei 230009, China. He is now with the State Key Laboratory of Information Photonics and Optical Communications, Beijing University of Posts and Telecommunications, Beijing 100876, China (e-mail: wangxiang2022s@163.com).

Minghua Cao is with the School of Computer and Communication, Lanzhou University of Technology, Lanzhou 730050, China (e-mail: caominghua@lut.edu.cn).

Digital Object Identifier 10.1109/TCOMM.2025.3647757

To improve the performance while addressing practical implementation constraints, one line of research is to use concatenated FEC schemes, which use SD-FEC codes as inner codes and HD-FEC codes as outer codes. The inner code, designed such that its decoder fulfils a given complexity constraint, is used for error reduction at the input of the outer HD decoder, while the outer HD decoder then lowers the error rate to the desired target. In recent years, the concatenated schemes of SCC and LDPC [2], [23], SCC and Hamming code (using SD decoding algorithm) [2], [24], SCC and polar code [25], KP4 (i.e., RS (528,514)) and single-parity-check code [26], zipper-LDPC code and zipper-BCH code [27] have been reported. These concatenated schemes show similar coding gains to those of existing SD-FEC schemes with significant complexity reduction. In addition, using mixed component codes (one is SD-FEC code and the other is HD-FEC code) is also proposed for the product-like codes enabling smaller decoding window size and faster convergence speed [28], [29].

Another promising FEC solution to strike a good balance between performance and complexity is hybrid decoding schemes, which aims to improve HD decoding performance with the help of channel soft information, i.e., log-likelihood ratios (LLRs) [30]. In [31], an iterative BDD with scaled reliability (iBDD-SR) is proposed, which makes hard decisions based on the reliabilities obtained by combining the properly scaled BDD outputs and the channel LLRs. In [32], the scaled reliability principle is extended to product codes based on generalized minimum distance decoding (GMDD) to update the LLRs at each iteration for finding the ordered least reliable bits. Further, a binary message-passing based on error-and-erasure decoding (BEE) algorithm is proposed by using an improved iBDD-SR algorithm (in which the combine rule is optimized via density evolution), showing a significant performance gain with respect to iterative BDD [33].

In [34] and [35], a class of soft-aided bit-marking (SABM) algorithm is proposed for SCCs to prevent miscorrections and to enhance the error-correcting capability of BDD by marking bits into three reliability levels according to the channel LLRs. Although better performance has been shown than iBDD-SR and BEE, there still is a large space for performance improvement if an effective update algorithm is proposed to eliminate the effect of erroneous marking during the iteration decoding process. In [36], a variant of SABM and AD algorithm is proposed with dynamic reliability score (DRS), where DRS reflects the reliability of a bit from both its channel LLR and its behavior during decoding. Besides, the work in [37] proposes a soft-aided HDD algorithm based on ordered statistics decoding (OSD), which involves reprocessing the HD decoding result of the reliable information set to generate a candidate codeword list.

Although the above mentioned methods proposed in [31], [32], [33], [34], [35], [36], and [37] are all hybrid decoding schemes, there is significant difference on the complexity, depending on how the LLRs get involved in the decoding process. The algorithms based on scaled reliability [31], [32] and BEE [33] always need channel LLRs during the decoding process (to calculate the scaled reliabilities based on the

channel LLRs or to sort the scaled reliabilities to erase the least reliable bits for each component codeword at every iteration). By contrast, the SABM-based [34], [35], [36] and OSD-based decoding algorithms [37] only require the LLRs once at the initial step for marking bits or DRS assignment or Gaussian elimination, and thus no need to store LLRs. Therefore, inspired by the definition of the hybrid degree for hybrid vehicles [38], we roughly divide the hybrid decoding algorithms into *strongly-hybrid* and *weakly-hybrid* two categories, based on whether the LLRs are always involved in the decoding process. The former always requires LLRs during the decoding process, while the latter only performs one-shot utilization of LLRs. Hence, the proportion of complexity resulting from LLR-relevant operations in the total complexity of the decoding algorithm for strongly-hybrid cases is intuitively much higher than that for weakly-hybrid cases. The strictly accurate definition of hybrid degree for the HD/SD hybrid error-correction decoding algorithms is interesting to be studied in future.

In this paper, we propose a *weakly-hybrid* HD/SD decoding algorithm for SCCs based on multi-level bit-marking (MLBM), where the LLRs are only used to perform one-shot bit marking when the bits are received from the channel. The proposed MLBM algorithm belongs to the SABM family that performs HD decoding with the help of marked bits (MBs), achieving superior performance with more reliability levels, simple but effective MB update mechanism and bit-flipping priority, compared to the SABM [34] and improved SABM (iSABM) [35] algorithms. The key contributions of the paper are summarized as follows:

- An MLBM decoding algorithm is proposed for SCCs, in which the HD-estimated bits are marked as four levels, i.e., highly reliable bits (HRBs), reliable bits (RBs), unreliable bits (UBs) and highly unreliable bits (HUBs), according to the channel LLRs using three thresholds. The proposed algorithm prevents BDD miscorrections by checking conflicts with HRBs and decodes those cases with too many errors via flipping HUBs and UBs. The decoding results will in turn help the decoder to determine the priority of the flipping bits and to update the marked bits in time.
- Parameter optimization of the proposed MLBM decoding algorithm is carried out in detail through numerical simulations. In particular, the prediction of the optimal marking thresholds in theory has been derived based on the probability distribution functions (PDFs) of the LLRs.
- Performance analysis of the proposed MLBM algorithm are conducted in detail, which show up to 1.22 dB gain with steeper waterfall region and lower error floor, when compared to conventional SCCs using HD decoding.
- A qualitative complexity analysis of the proposed MLBM decoding algorithm for SCCs is performed, including the increased complexity caused by the additional BDDs and the increased memory usage.
- The effectiveness of the proposed MLBM algorithm for PCs has been demonstrated, which shows up to 0.78 dB gain with respect to conventional PC using HD decoding.

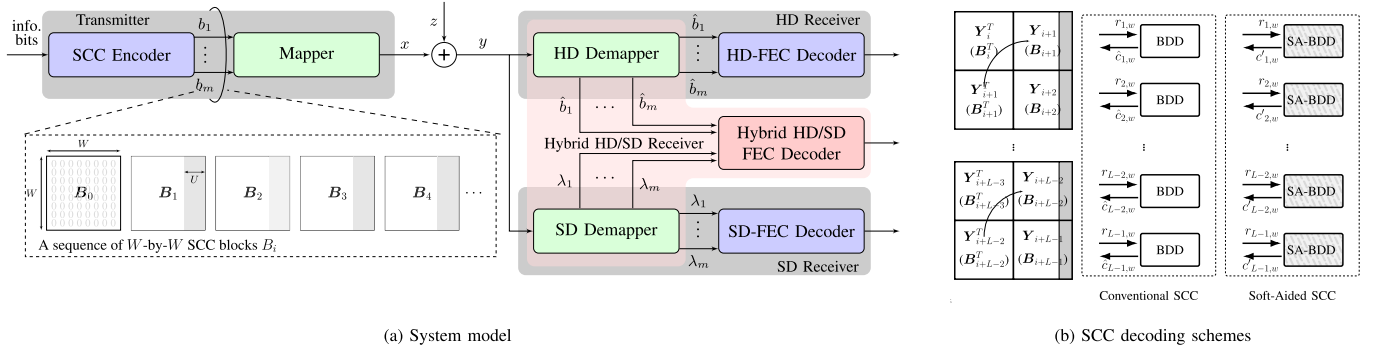


Fig. 1. System model considered in this paper (a) and an illustration of SCC decoding schemes (b).

The remainder of the paper is organized as follows. In Sec. II, we introduce system model and review the principles of conventional SCCs. In Sec. III, we describe the proposed MLBM decoding algorithm for SCCs. The parameter optimization is given in Sec. IV and performance analysis is shown in Sec. V, while the complexity of MLBM is discussed in Sec. VI. In Sec. VII, the proposed MLBM algorithm has been extended to PCs. Finally, Sec. VIII concludes this paper.

Notation: Boldface letters are used to denote the matrix and vector, e.g., $\mathbf{B}$ and $\mathbf{e}$, while the letters with math calligraphy, e.g., $\mathcal{I}$, are used to denote the set. $\mathcal{I}(i)$ and $e(i)$ represent the i -th element in set $\mathcal{I}$ and vector $\mathbf{e}$, respectively. The expression, for example, $\mathcal{P} = \{i : e(i) = 1\}$ returns the positions where $e(i) = 1$. $|\cdot|$ returns the absolute value when its argument is a number, and returns the number of elements if its argument is a matrix, or a vector, or a set.

II. SYSTEM MODEL AND SCCS

A. System Model

The system model of this paper is depicted in Fig. 1 (a). Information bits are encoded by an SCC encoder and are mapped to symbol x every m bits $\{b_1, b_2, \dots, b_m\}$ at each time instant via a mapper. The symbol x is taken from an equally-spaced M -ary pulse amplitude modulation (PAM) constellation with M points $\mathcal{S} = \{s_1, s_2, \dots, s_M\}$, where $M = 2^m$. After transmission over an additive white Gaussian noise (AWGN) channel, the received symbol is given by $y = x + z$, where $z \sim \mathcal{N}(0, N_0)$ and $\mathbb{E}[x^2] = 1$ ($\mathbb{E}[\cdot]$ denotes expectation). The SNR is defined as $\gamma = 1/N_0$ and we have $y \sim \mathcal{N}(x, N_0)$.

For conventional HD receiver, the received signal y is directly converted to binary bits $\hat{b}_1, \dots, \hat{b}_m$ via an HD demapper and is decoded by an HD-FEC decoder, for example conventional SCCs. Therefore, the main feature of HD receiver is its low complexity, but with significant loss of channel information. By contrast, for conventional SD receiver, the received signal y will be converted to channel LLRs $\lambda_1, \dots, \lambda_m$, given by

$$\lambda_k = \sum_{b \in \{0,1\}} (-1)^{\bar{b}} \log \sum_{i \in \mathcal{I}_{k,b}} \exp \left(-\frac{(y - s_i)^2}{2N_0} \right), \quad (1)$$

with $k = 1, 2, \dots, m$, where $\bar{b}$ denotes bit negation, the set $\mathcal{I}_{k,b}$ enumerates all constellation points in $\mathcal{S}$ where the k th

bit is b . According to (1), we can know that $\lambda_k > 0$ means that there is a larger probability that the received bit is 1, while $\lambda_k < 0$ means that there is a larger probability that the received bit is 0. The larger $|\lambda_k|$ is, the more reliable the result will be.

Particularly, the LLR for 2-PAM is given by

$$\lambda = \log \frac{\exp(-(\gamma/2)(y-1)^2)}{\exp(-(\gamma/2)(y+1)^2)} = 2\gamma y. \quad (2)$$

It can be seen that for 2-PAM case, λ is a Gaussian random variable, i.e., $\lambda \sim \mathcal{N}(2\gamma x, 4\gamma)$, where $x = 1$ for bit 1 and $x = -1$ for bit 0. The PDFs of the LLRs can be written as

$$\begin{cases} p_0(\lambda) = \frac{1}{2\sqrt{2\pi\gamma}} \exp \left(-\frac{(\lambda + 2\gamma)^2}{8\gamma} \right), & \text{if } x = -1 \\ p_1(\lambda) = \frac{1}{2\sqrt{2\pi\gamma}} \exp \left(-\frac{(\lambda - 2\gamma)^2}{8\gamma} \right), & \text{if } x = 1 \end{cases} \quad (3)$$

The SD-FEC decoder will use the LLRs for FEC decoding to recover the transmitted bits. With more accurate channel information, SD receiver offers much better performance than HD receiver, but at the expense of complexity.

To tradeoff the complexity and performance, this paper considers a hybrid receiver. In this scheme, HD and SD demappers are both needed to obtain the HD-estimated bits and LLRs, respectively, which will all be sent to the FEC decoder to perform hybrid decoding for error correction. The key idea for the hybrid decoder is to use soft information to help HD decoding, and its performance and complexity depend on the way soft information is used.

B. SCCs

SCCs are considered in this paper, due to their popularity in optical communications in recent years. SCC combines ideas from recursive convolutional coding and block coding, which consists of a sequence of binary blocks $\{B_1, B_2, B_3, \dots\}$, as the bottom-left of Fig. 1 (a) shows. Each block B_i contains $W \times W$ bits, while the rightmost $W \times U$ bits are parity bits, for $i > 0$. The block B_0 is initialized to be an all-zero block. For $\forall i > 0$, each row of the matrix $[B_{i-1}^T B_i]$ is a valid codeword in a component code $\mathcal{C}$ (an extended or shortened BCH code is often used), where B_{i-1}^T is the matrix transpose of B_{i-1} . Let (n_c, k_c, t_c) be the parameters of a BCH code, where n_c is the codeword length (an even number), k_c is the information

length, and t_c is the error-correcting capability. The code rate of SCC is given by

$$R = 1 - U/W = 2k_c/n_c - 1, \quad (4)$$

where $W = n_c/2$ and $U = n_c - k_c$.

SCC decoding is performed using sliding window fashion each covering L received SCC blocks. Fig. 1 (b) gives an illustration of SCC decoding in the i th window, which includes blocks $\{\mathbf{Y}_i, \mathbf{Y}_{i+1}, \dots, \mathbf{Y}_{i+L-1}\}$ that $\mathbf{Y}_i$ corresponds to the transmitted SCC block $\mathbf{B}_i$ for $i \geq 1$ and $\mathbf{Y}_0$ is an all-zero matrix. In conventional SCC decoding, BDD is used to iteratively decode each row of two neighbor SCC block from the bottom to the top of the window. When the maximum number of iterations G is reached or all the errors have been corrected, the sliding window outputs block $\mathbf{Y}_i$ and moves to cover the next L blocks $\{\mathbf{Y}_{i+1}, \mathbf{Y}_{i+2}, \dots, \mathbf{Y}_{i+L}\}$ and repeats the decoding process in the last window. This process will persist until all the received bits have been decoded.

Let $\mathbf{c}_{j,w} \in \mathcal{C}$ and $\mathbf{r}_{j,w}$ be the transmitted codeword and received sequence, respectively, in the w th row of matrix $[\mathbf{Y}_{i+j-1}^T \mathbf{Y}_{i+j}]$ of the i th window, where $j \in \{1, 2, \dots, L-1\}$ and $w \in \{1, 2, \dots, W\}$. The output $\hat{\mathbf{c}}_{j,w}$ of BDD with t_c -error correcting capability can be written as

$$\hat{\mathbf{c}}_{j,w} = \begin{cases} \mathbf{c}_{j,w}, & \text{if } d(\mathbf{r}_{j,w}, \mathbf{c}_{j,w}) \leq t_c \\ \bar{\mathbf{c}}_{j,w}, & \text{if } d(\mathbf{r}_{j,w}, \mathbf{c}_{j,w}) > t_c \text{ and } d(\mathbf{r}_{j,w}, \bar{\mathbf{c}}_{j,w}) \leq t_c \\ \mathbf{r}_{j,w}, & \text{otherwise} \end{cases} \quad (5)$$

where $\bar{\mathbf{c}}_{j,w}$ is another codeword in $\mathcal{C}$, $d(\cdot, \cdot)$ represents the Hamming distance. The first two cases given in (5) both correspond to BDD success, but the second one is in fact a miscorrection (also known as decoding error). Mis-correction will introduce extra errors, and thus cause performance degradation, particularly in the iterative decoding fashion. The third case in (5) corresponds to BDD failure that will change nothing.

To yield more gains, several hybrid decoding algorithm are used to prevent miscorrections and/or to enhance the error-correcting capability of BDD, e.g., iBDD-SR [31], BEE [33], SABM [34] and iSABM [35]. Differently from conventional SCC decoding, these algorithms use soft-aided BDD (SA-BDD) to iteratively decode each row of two neighbor SCC blocks, as shown in Fig. 1 (b) (where $\mathbf{c}'_{j,w}$ is the output of the SA-BDD component decoder corresponding to $\mathbf{r}_{j,w}$). In next section, we will introduce our proposed new hybrid decoding algorithm for SCC in detail.

III. THE PROPOSED MLBM DECODING ALGORITHM

A. Main Algorithm Routine

The main routine of the proposed MLBM decoding algorithm is shown in Fig. 2, where the black and blue lines represent the decoding data flow and the MB data flow, respectively. Symbol $s_{j+1,w}$ denotes the decoding status corresponding to $\mathbf{r}_{j,w}$. The value of $s_{j+1,w}$ indicates four types of decoding status, including

- 0: decoding success with high confidence,

- 1: decoding success,
- 2: identified mis-correction,
- 3: decoding failure.

Let $\mathbf{S}$ denote an $(L+1) \times W$ decoding status matrix, whose elements from the 2th row to the L th row are given by $s_{j+1,w}$ with $j = 1, 2, \dots, L-1$ and $w = 1, 2, \dots, W$. Particularly, the first and last rows of $\mathbf{S}$ are initialized to all 3. The MBs for the HD-estimated bits include HRBs, RBs, UBs, HUBs four types.

Like conventional SCC decoding, BDD will be used to decode the received sequence $\mathbf{r}_{j,w}$ firstly and outputs the decoding result $\hat{\mathbf{c}}_{j,w}$ and detected error pattern $\mathbf{e}_{j,w}$ ($\mathbf{e}_{j,w} = \hat{\mathbf{c}}_{j,w} \oplus \mathbf{r}_{j,w}$). As the block closer to the front of the window has been decoded more times (because of the decoding in the previous windows), there are fewer errors leading to smaller mis-correction probability. Therefore, a position threshold K_{MD} ($0 \leq K_{MD} < L$) is defined to indicate where to start to perform mis-correction detection (MD). If BDD declares decoding success and $j \leq K_{MD}$, the decoder will directly trust BDD result that outputs $\mathbf{c}'_{j,w} = \hat{\mathbf{c}}_{j,w}$ and the decoding status is $s_{j+1,w} = 0$ (①, i.e., the first path in Fig.2). If $j > K_{MD}$, MLBM will perform mis-correction detection for success cases of BDD, assisted by HRBs. If not a mis-correction, the MLBM decoder outputs $\mathbf{c}'_{j,w} = \hat{\mathbf{c}}_{j,w}$ and updates the decoding status to 0 or 1, i.e., $s_{j+1,w} = 0$ or $s_{j+1,w} = 1$ (②, i.e., the second path in Fig.2). The specific value of $s_{j+1,w}$ in this case depends on the conflicting situation with HRBs, which will be explained in Sec. III-C. For identified miscorrections and BDD failures, bit flipping (BF) will be performed to try to correct some errors, which are HUBs and UBs, in $\mathbf{r}_{j,w}$. The output sequence $\mathbf{r}'_{j,w}$ of BF will be sent to the BDD decoder again followed by MD for BDD success. Similarly, if the output of the second BDD trial is not a mis-correction, the MLBM decoder will accept the decoding result $\tilde{\mathbf{c}}_{j,w}$, i.e., $\mathbf{c}'_{j,w} = \tilde{\mathbf{c}}_{j,w}$, and the decoding status is $s_{j+1,w} = 0$ or $s_{j+1,w} = 1$ (③, i.e., the third path in Fig.2). By contrast, if the second BDD output is regarded as a mis-correction, the decoder will change nothing with $\mathbf{c}'_{j,w} = \mathbf{r}_{j,w}$ and the decoding status is $s_{j+1,w} = 2$ (④, i.e., the fourth path in Fig.2). For the decoding failure in the second BDD trial, the MLBM decoder outputs $\mathbf{c}'_{j,w} = \mathbf{r}_{j,w}$ and the decoding status is $s_{j+1,w} = 3$ (⑤, i.e., the fifth path in Fig.2). Once the decoding corresponding to $\mathbf{r}_{j,w}$ is done, the decoder will update the marked information in time, according to the decoding result and decoding status.

As can be seen from Fig. 2, to make the the proposed MLBM decoding algorithm work well, there are four important parts: *bit marking*, *miscorrection detection*, *bit flipping*, and *marked bit update*.

B. Bit Marking

Bit marking is the first step for the proposed MLBM algorithm when the decoder receives an SCC block from the channel. To divide the HD-estimated bits into HRB, RB, UB and HUB four levels, three reliability thresholds δ_1 , δ_2 and δ_3 are used, where $\delta_1 > \delta_2 > \delta_3 \geq 0$. According to the value of

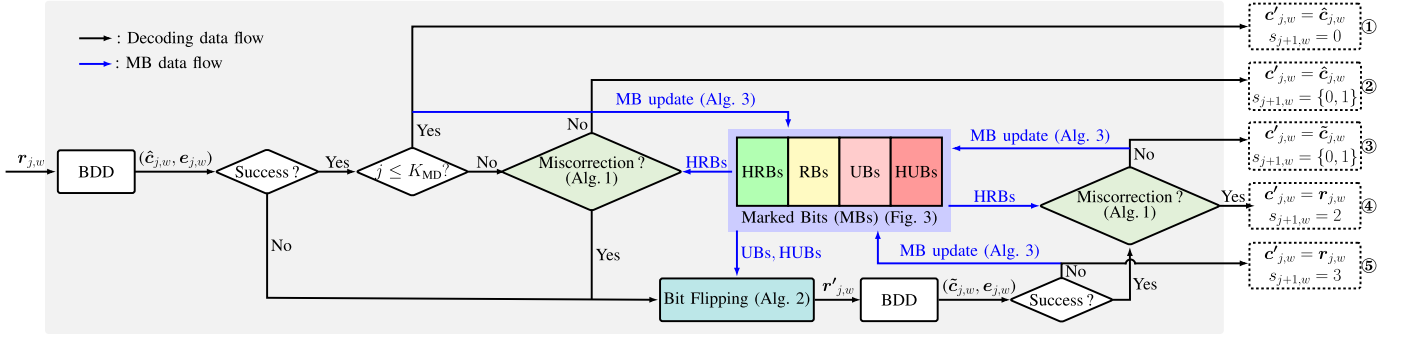


Fig. 2. The main routine of the proposed MLBM algorithm. The black lines indicate the decoding data flow, while the blue lines indicate the MB data flow.

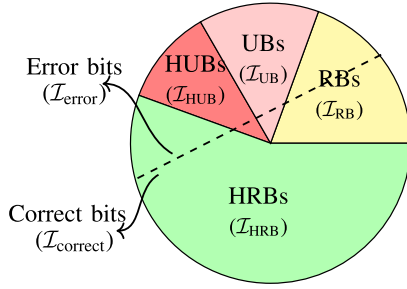


Fig. 3. An illustration to indicate the relationship between the marked bits and the received bits.

$|\lambda_k|$, the marking result for a recovered bit $\hat{b}_k$ is given by

$$\begin{cases} \text{HRB,} & \text{if } |\lambda_k| \geq \delta_1 \\ \text{RB,} & \text{if } \delta_2 \leq |\lambda_k| < \delta_1 \\ \text{UB,} & \text{if } \delta_3 \leq |\lambda_k| < \delta_2 \\ \text{HUB,} & \text{if } |\lambda_k| < \delta_3 \end{cases} \quad (6)$$

When bit marking is done, the soft information is not needed anymore, and thus no need to be stored.

From (6), we know that HRBs are the most reliable bits, and HUBs are the most unreliable bits, then are UBs. Ideally, we hope that HRBs are all correct bits to correctly identify miscorrections, and HUBs are all errors to perfectly correct errors via BF. However, the practical case is that there are probabilities that HUB is a correct bit and HRB is an error bit. Fig. 3 illustrates the relationship between the marked bits and the received bits. Notations $\mathcal{I}_{HRB}$, $\mathcal{I}_{RB}$, $\mathcal{I}_{UB}$ and $\mathcal{I}_{HUB}$ represent the sets of HRBs, RBs, UBs and HUBs, respectively. The left and right sides of the dashed line represent the received error bits (denoted by $\mathcal{I}_{error}$) and correct bits (denoted by $\mathcal{I}_{correct}$), respectively. $\mathcal{I}_{PE}$ will be used to denote the union of $\mathcal{I}_{UB}$ and $\mathcal{I}_{HUB}$, i.e., $\mathcal{I}_{PE} = \mathcal{I}_{HUB} \cup \mathcal{I}_{UB}$. These sets satisfy such a relationship $\mathcal{I}_{HRB} \cup \mathcal{I}_{RB} \cup \mathcal{I}_{UB} \cup \mathcal{I}_{HUB} = \mathcal{I}_{correct} \cup \mathcal{I}_{error}$. Ideally, we hope that all the errors are in the set $\mathcal{I}_{PE}$.

Let α_{HUB} and α_{PE} be the probabilities that the bit in $\mathcal{I}_{HUB}$ and $\mathcal{I}_{PE}$ is an error, β_{HUB} and β_{PE} be the probabilities that an error bit is marked as HUB and PE, respectively. And let α_{HRB} be the probability that an HRB is a correct bit, β_{HRB} be the probability that a correct bit is marked as an HRB, we have

Algorithm 1 Miscoction Detection

Input: $e_{j,w}, \mathcal{H}$

- 1 $s_{j+1,w} = 0$;
- 2 $\mathcal{P} = \{i : e_{j,w}(i) = 1\}$
- 3 **for** $i = 1, 2, \dots, |\mathcal{P}|$ **do**
- 4 **if** $\mathcal{P}(i) \in \mathcal{I}_{HRB}$ **then**
- 5 $\mathcal{H}(\mathcal{P}(i)) = \mathcal{H}(\mathcal{P}(i)) + 1$;
- 6 **if** $\mathcal{H}(\mathcal{P}(i)) > T_{HRB}$ **then**
- 7 $s_{j+1,w} = 1$;
- 8 $\mathcal{H}(\mathcal{P}(i)) = 0$;
- 9 **break**;
- 10 **else**
- 11 $s_{j+1,w} = 2$;

Output: $s_{j+1,w}$

$$\begin{cases} \alpha_{HUB} = |\mathcal{I}_{HUB} \cap \mathcal{I}_{error}| / |\mathcal{I}_{HUB}| \\ \beta_{HUB} = |\mathcal{I}_{HUB} \cap \mathcal{I}_{error}| / |\mathcal{I}_{error}| \\ \alpha_{PE} = |\mathcal{I}_{PE} \cap \mathcal{I}_{error}| / |\mathcal{I}_{PE}| \\ \beta_{PE} = |\mathcal{I}_{PE} \cap \mathcal{I}_{error}| / |\mathcal{I}_{error}| \\ \alpha_{HRB} = |\mathcal{I}_{HRB} \cap \mathcal{I}_{correct}| / |\mathcal{I}_{HRB}| \\ \beta_{HRB} = |\mathcal{I}_{HRB} \cap \mathcal{I}_{correct}| / |\mathcal{I}_{correct}| \end{cases} \quad (7)$$

The values of α_{HUB} and α_{PE} will determine the probability of flipping error bits via BF. Therefore, the values of α_{HUB} and α_{PE} are expected to be as large as possible. On the other hand, the value of α_{HRB} will have a straightforward impact on the accuracy of the detection of miscorrection, which will be explained in Sect. III-C. Ideally, we hope to have $\alpha_{HUB} \rightarrow 1$, $\alpha_{PE} \rightarrow 1$, and $\alpha_{HRB} \rightarrow 1$. To avoid too many variables, $\mathcal{I}_{HRB}$, $\mathcal{I}_{RB}$, $\mathcal{I}_{UB}$, $\mathcal{I}_{HUB}$, and $\mathcal{I}_{PE}$ will only represent the marked bits corresponding to $r_{j,w}$ in the rest of the paper.

C. Miscoction Detection

Algorithm 1 shows the pseudocode for miscorrection detection whose input parameters are error pattern $e_{j,w}$ and a $1 \times 2W$ vector $\mathcal{H}$ recording the number of conflicts between BDD outputs and HRBs for $r_{j,w}$ (if a bit in $r_{j,w}$ is not an HRB, the corresponding element value in $\mathcal{H}$ is 0). With the detected error pattern $e_{j,w}$, MD will find where the errors are, i.e., the positions of '1's in $e_{j,w}$. Let $\mathcal{P} = \{i : e_{j,w}(i) = 1\}$ denote the detected error positions. Since there is a high probability that HRBs are correct bits, MD will identify

miscorrections by checking whether the detected errors are involved in any HRBs. If there is any conflict between BDD output and HRBs (i.e., $\exists i \in [1, |\mathcal{P}|], \mathcal{P}(i) \in \mathcal{I}_{\text{HRB}}$), we will have $\mathcal{H}(\mathcal{P}(i)) = \mathcal{H}(\mathcal{P}(i)) + 1$. If the number of conflicts for HRB exceeds a threshold T_{HRB} ($T_{\text{HRB}} > 0$), i.e., $\mathcal{H}(\mathcal{P}(i)) > T_{\text{HRB}}$, it indicates that this HRB is not reliable enough and should be removed from $\mathcal{I}_{\text{HRB}}$ set. That is, we will trust BDD output with decoding status $s_{j+1,w} = 1$. If $\mathcal{H}(\mathcal{P}(i)) \leq T_{\text{HRB}}$, BDD output will be regarded as a miscorrection and the decoding status is $s_{j+1,w} = 2$. For $\forall i$, if $\mathcal{P}(i) \notin \mathcal{I}_{\text{HRB}}$ (indicating that there is no conflict between the BDD output and the HRBs), BDD output will be trusted, and we have $s_{j+1,w} = 0$.

D. Bit Flipping

Algorithm 2 shows the pseudocode for bit flipping, where f_g is the number of flipped bits in the g th iteration, $g = 1, 2, \dots, G$. Vector $\mathbf{f} = [f_1, \dots, f_g, \dots, f_G]$ will be used to indicate the number of flipped bits at each iteration. Let set $\mathcal{V}_{s=\{2,3\}} = \{i : s_{j,:}(i) = (2 \parallel 3)\} \cup \{W + \{i : s_{j+2,:}(i) = (2 \parallel 3)\}\}$, where $s_{j,:}$ and $s_{j+2,:}$ denote the j th and $(j+2)$ th rows of $\mathbf{S}$, respectively. According to the MBs and decoding statuses related to the bits in $\mathbf{r}_{j,w}$, three priority levels are assigned for BF to flip bits, which are given by

- *Highest priority*: set $\mathcal{R}_0 = \mathcal{I}_{\text{HUB}} \cap \mathcal{V}_{s=\{2,3\}}$, in which the bit is an HUB and the related sequences $\mathbf{r}_{j-1,w}$ (for the first half of bits in $\mathbf{r}_{j,w}$) or $\mathbf{r}_{j+1,w}$ (for the second half of bits in $\mathbf{r}_{j,w}$) exists errors, i.e., the decoding status is a miscorrection or an failure.
- *Medium priority*: set $\mathcal{R}_1 = \mathcal{I}_{\text{PE}} \cap \mathcal{V}_{s=\{2,3\}}$, in which the bit is an HUB or UB and the related sequences $\mathbf{r}_{j-1,w}$ or $\mathbf{r}_{j+1,w}$ exists errors.
- *Lowest priority*: set $\mathcal{R}_2 = \mathcal{I}_{\text{PE}}$, in which the bit is an HUB or UB.

Algorithm 2 Bit Flipping

Input: $\mathbf{r}_{j,w}, \mathcal{R}_0, \mathcal{R}_1, \mathcal{R}_2, f_g$

```

1  $\mathbf{r}'_{j,w} = \mathbf{r}_{j,w}$ ;
2 if  $|\mathcal{R}_0| \geq f_g$  then
3   Randomly flip  $f_g$  bits of  $\mathbf{r}'_{j,w}$  located in  $\mathcal{R}_0$ ;
4 else
5   if  $|\mathcal{R}_1| \geq f_g$  then
6     Randomly flip  $f_g$  bits of  $\mathbf{r}'_{j,w}$  located in  $\mathcal{R}_1$ ;
7   else
8     if  $|\mathcal{R}_2| \geq f_g$  then
9       Randomly flip  $f_g$  bits of  $\mathbf{r}'_{j,w}$  located in  $\mathcal{R}_2$ ;
Output:  $\mathbf{r}'_{j,w}$ 

```

The three sets satisfy such relationships: $\mathcal{R}_0 \subseteq \mathcal{R}_1 \subseteq \mathcal{R}_2$.

Based on the designed priority rule, BF will first randomly flip f_g bits from $\mathcal{R}_0$ if $|\mathcal{R}_0| \geq f_g$. If $|\mathcal{R}_0| < f_g$ but $|\mathcal{R}_1| \geq f_g$, BF will randomly flip f_g bits from $\mathcal{R}_1$. Otherwise, BF will randomly flip f_g bits from $\mathcal{R}_2$ if $|\mathcal{R}_2| \geq f_g$, or do nothing if $|\mathcal{R}_2| < f_g$.

Particularly, we find that for miscorrection, $d_0 - t_c - w_H(\mathbf{e}_{j,w})$ (which is used for SABM [34] and iSABM [35]) is still the optimal number of flipped bits for this algorithm at each iteration, where d_0 is the minimum Hamming distance between any two codewords in $\mathcal{C}$ and $w_H(\cdot)$ denotes the Hamming weight. While the optimal number of flipped bits for BDD failure at each iteration is different, which will be shown in Sec. V-A via numerical simulation. In the remaining contents, we will use vector $\mathbf{f}_{\text{miscorrection}}$ and $\mathbf{f}_{\text{failure}}$ to respectively indicate the number of flipped bits at each iteration, where $\mathbf{f}_{\text{miscorrection}} = d_0 - t_c - w_H(\mathbf{e}_{j,w})$.

Note that with the definition of the three priority levels among the bits in UBs and HUBs, the whole bits are actually divided into five levels (depending on the LLRs and the decoding status), which are the sets $\mathcal{I}_{\text{HRB}}$, $\mathcal{I}_{\text{RB}}$, $\mathcal{I}_{\text{PE}} - \mathcal{R}_1$, $\mathcal{R}_1 - \mathcal{R}_0$ and $\mathcal{R}_0$ (in descending order by reliability). In addition, the operation that subsequently flipping bits from different reliability classes is somewhat similar to GMDD [39] when multiple reliability classes are adopted. However, there exists a significant difference that GMDD has no idea how many errors in the received sequence, while MLBM knows that there are more than t_c errors (as decoding failure or miscorrection occurs in the first BDD trial), when bit flipping is performed. As a result, GMDD tries $t_c + 1$ error and erasure decodings (each contains two BDDs) by erasing $\{d_0 - 1, d_0 - 3, \dots, d_0 - 2t_c - 1\}$ least reliable bits, and selects the final decoding result using the generalized minimum distance decoding criterion. In contrast, according to the decoding status of the first BDD trial, MLBM purposefully executes different bit flipping strategies for decoding failures and miscorrections, allowing the errors in the received sequence to have a high probability to be decoded with only one extra BDD trial.

E. Marked Bit Update

To significantly improve the performance of SCC, marked bit update is key. On the one hand, updating the marked HUBs, UBs and RBs in time can avoid the correctly decoded bits to be reverted to wrong bits, due to BFs or BDDs in the following decodings. On the other hand, updating the unreliable HRBs can avoid identifying miscorrection mistakenly, and gives opportunity to the decoder to correct errors in HRBs.

Algorithm 3 shows how to update the marked bits in $\mathbf{r}_{j,w}$ according to its decoding status $s_{j+1,w}$. Let $\mathcal{V}_{s=\{0,1\}} = \{i : s_{j,:}(i) = (0 \parallel 1)\} \cup \{W + \{i : s_{j+2,:}(i) = (0 \parallel 1)\}\}$. The details of MBU are as follows.

- For $s_{j+1,w} = 0$, two cases are considered. One is the decoded sequence with no errors and the other is the sequence with errors. For the first case, since there is a large probability that the decoding is indeed correct, the decoder will update all the bits in $\mathcal{V}_{s=\{2,3\}}$ to RBs. This will protect the bits from being flipped by BF in the following decodings. For the latter case, as the detected errors are from the successfully decoded sequence, i.e., $\mathcal{P} \subseteq \mathcal{V}_{s=\{0,1\}}$, it is possible that the decoding is an unidentified miscorrection. Therefore, the decoder will update the detected error bits to HUBs. As a result,

this gives a highest priority for BF to revert the bits for correction in the following decodings.

- For $s_{j+1,w} = 1$, as mentioned in Sec. III-C, it indicates the conflicting HRB is not reliable enough (since it conflicts with too many BDDs), and thus the corresponding decoding result is accepted. In this case, the decoder will update the conflicting HRBs (which is also an error) to RBs. In case that BDD is indeed a successful decoding, the remaining detected errors will be updated to RBs as well, to prevent from being flipped during the subsequent decodings by BF.

Algorithm 3 Marked Bit Update

Input: $s_{j+1,w}, \mathcal{P}, \mathcal{I}_{\text{HRB}}, \mathcal{I}_{\text{RB}}, \mathcal{I}_{\text{UB}}, \mathcal{I}_{\text{HUB}}, \mathcal{V}_s = \{0,1\}, \mathcal{V}_s = \{2,3\}$

```

1 if  $s_{j+1,w} = 0$  then
2   if  $|\mathcal{P}| = 0$  then
3      $\mathcal{I}_{\text{RB}} = \mathcal{I}_{\text{RB}} \cup \mathcal{V}_s = \{2,3\}$ ;
4   else
5     if  $\mathcal{P} \subseteq \mathcal{V}_s = \{0,1\}$  then
6        $\mathcal{I}_{\text{HUB}} = \mathcal{I}_{\text{HUB}} \cup \mathcal{P}$ ;
7 else
8   if  $s_{j+1,w} = 1$  then
9      $\mathcal{I}_{\text{RB}} = \mathcal{I}_{\text{RB}} \cup \mathcal{P}$ ;
10  else
11    if  $s_{j+1,w} = 3$  then
12      Add the flipped bits via BF to set  $\mathcal{I}_{\text{UB}}$ ;

```

Output: $\mathcal{I}_{\text{HRB}}, \mathcal{I}_{\text{RB}}, \mathcal{I}_{\text{UB}}, \mathcal{I}_{\text{HUB}}$

- For $s_{j+1,w} = 3$, the decoder will update all the flipped bits via BF to UBs. This will give priority to the decoder to flip other HUBs in the following decoding, once the current decoding fails.
- Otherwise, the decoder will keep the marked bits unchanged.

IV. PARAMETERS' OPTIMIZATION

Optimizing the marking thresholds δ_1 , δ_2 and δ_3 is the primary task for the proposed algorithm to yield additional coding gain. Therefore, in this section, we will first optimize the three marking thresholds δ_1 , δ_2 and δ_3 via numerical simulation and theoretical prediction both. Then we optimize the other parameters including f_{failure} (the number of flipping bits for BDD failures), K_{MD} (the position threshold for performing MD) and T_{HRB} (the unreliable HRB threshold). The optimization is performed for SCC based on BCH (256,239,2) with modulation format of 2-PAM. The decoding window size is $L = 9$ and the maximum of iteration times is $G = 7$.

A. Marking Thresholds δ_1 , δ_2 , δ_3

1) *Numerical Simulation:* Numerical simulation is a straightforward way to optimize the parameters. In this way, the three thresholds normally should be optimized jointly to find the optimal values, however, this will make the optimization too complex. For simplicity, we first optimize δ_1 and δ_2 together by setting $\delta_3 = 0$, and then optimize δ_3 . Other parameters are initialized as follow: $f_{\text{failure}} = [1, 1, 1, 1, 1, 1, 1]$, $K_{\text{MD}} = 2$ (which is the optimal value for iSABM [35]), and $T_{\text{HRB}} = 7$ (which is the maximum of iteration times).

Fig. 4 (a) shows the optimization results for δ_1 and δ_2 with 2-PAM at SNR of 6.3 dB (where the BER of the proposed

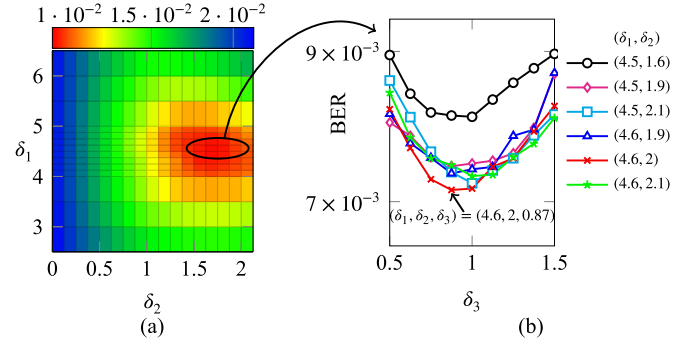


Fig. 4. Numerical optimization results for (a) δ_1 and δ_2 (while δ_3 is set to 0) and (b) δ_3 under certain (δ_1, δ_2) for SCC based on BCH (256,239,2) and 2-PAM at SNR of 6.3 dB.

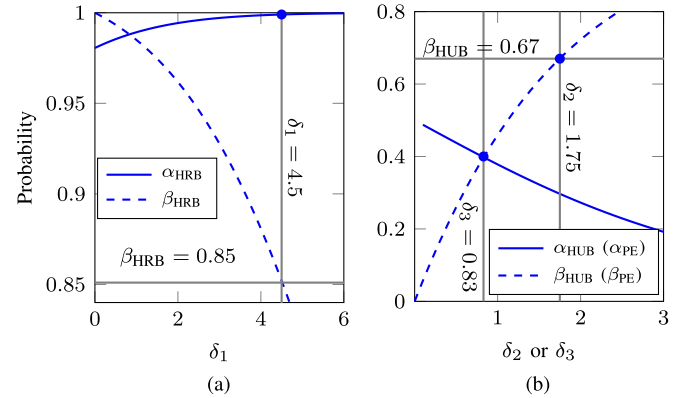


Fig. 5. The calculated values of (a) α_{HRB} , β_{HRB} , (b) α_{PE} and β_{PE} , α_{HUB} and β_{HUB} in theory against marking thresholds for 2-PAM at 6.3 dB SNR.

MLBM algorithm is around 10^{-3}). The color gradient from blue to red represents the reduction in BER. It shows that the region around $\delta_1 = 4.5$ and $\delta_2 = 1.75$ has the lowest BER, and larger or smaller δ_1 and δ_2 will lead to performance degradation. If δ_1 is too small, some error bits will be marked as HRBs, leading to mistakenly reject correct BDDs. By contrast, if δ_1 is too large, few correct bits will be marked as HRBs, which cannot help to identify the BDD miscorrections efficiently. For δ_2 , small value will make it hard to have enough UBs for flipping, while large value will it hard to flip the error bits, as too many correct bits will be mistakenly marked as UBs.

Fig. 4 (b) is the optimization result for δ_3 when the values for δ_1 and δ_2 are around their optimal ones. It shows that the optimized thresholds for δ_1 , δ_2 and δ_3 are 4.6, 2 and 0.87, respectively. The reason why larger or smaller δ_3 will cause performance degradation is the same as that for δ_2 , the only difference is that the HUBs marked via threshold δ_3 has higher priority than the UBs marked via threshold δ_2 for BF.

2) *Theoretical Prediction:* Let $\Psi_{b,[I_l, I_u]} = \int_{I_l}^{I_u} p_b(\lambda) d\lambda$ with $b \in \{0, 1\}$. For 2-PAM, we have $\Psi_{1,[I_l, I_u]} = \Psi_{0,[-I_u, -I_l]}$. In particular, $\Psi_{1,[-\infty, 0]}$ denotes the probability that the received bit is 0 when bit 1 is transmitted, while $\Psi_{0,[0, \infty]}$ denotes the probability that the received bit is 1 when bit 0 is transmitted. The six probabilities α_{HRB} , β_{HRB} , α_{PE} , β_{PE} , α_{HUB} , and β_{HUB}

can be theoretically written as

$$\begin{cases} \alpha_{\text{HRB}} = \Psi_{1, [\delta_1, \infty]} / (\Psi_{1, [\delta_1, \infty]} + \Psi_{1, [-\infty, -\delta_1]}) \\ \beta_{\text{HRB}} = \Psi_{1, [\delta_1, \infty]} / \Psi_{1, [0, \infty]} \\ \alpha_{\text{PE}} = \Psi_{1, [-\delta_2, 0]} / \Psi_{1, [-\delta_2, \delta_2]} \\ \beta_{\text{PE}} = \Psi_{1, [-\delta_2, 0]} / \Psi_{1, [-\infty, 0]} \\ \alpha_{\text{HUB}} = \Psi_{1, [-\delta_3, 0]} / \Psi_{1, [-\delta_3, \delta_3]} \\ \beta_{\text{HUB}} = \Psi_{1, [-\delta_3, 0]} / \Psi_{1, [-\infty, 0]} \end{cases} \quad (8)$$

According to (3), the values of α_{HRB} , β_{HRB} , α_{PE} , β_{PE} , α_{HUB} and β_{HUB} under different values of δ_1 , δ_2 and δ_3 are calculated. The results are shown in Fig. 5. From Fig. 5 (a), we can see that α_{HRB} increases as δ_1 increases. To avoid mistakenly marking the correct BDD as miscorrection, δ_1 should be large enough to make α_{HRB} approaching 1 for the proposed MLBM algorithm. However, the dashed curve in the figure shows that a larger δ_1 will lead to fewer HRB, limiting the capability of miscorrection detection. Therefore, the point $\delta_1 = 4.5$ (where α_{HRB} starts to approach 1) is chosen as the optimal one. At this point, $\beta_{\text{HRB}} = 85\%$.

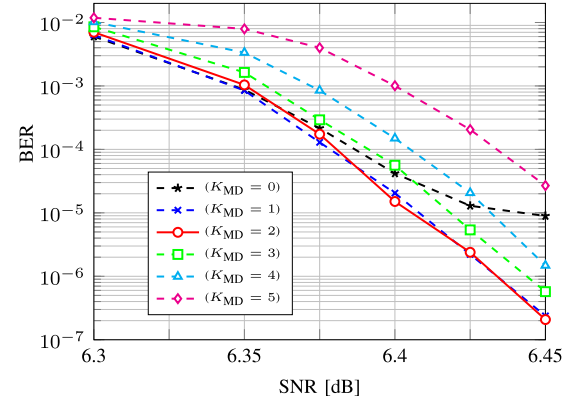
Fig. 5 (b) shows the value of α_{HUB} and β_{HUB} versus δ_3 . It can be seen that when δ_3 increases, α_{HUB} will decrease and β_{HUB} will increase. Having more errors in HUBs (i.e., larger β_{HUB}) and higher marking accuracy (i.e., larger α_{HUB}) are both important for the MLBM algorithm to quickly find enough errors via BF. Therefore, the intersection point of the two curves is the optimal value for δ_3 , i.e., $\delta_3 = 0.83$.

Note that the two curves in Fig. 5 (b) also indicate the relationship between α_{PE} , β_{PE} and δ_2 . To have opportunity to correct the cases with more than t_c errors via BF, there are at least $d(\mathbf{r}_{j,w}, \mathbf{c}_{j,w}) - t_c$ errors in the set $\mathcal{I}_{\text{PE}}$. Therefore, we have $\beta_{\text{PE}} \geq (d(\mathbf{r}_{j,w}, \mathbf{c}_{j,w}) - t_c) / d(\mathbf{r}_{j,w}, \mathbf{c}_{j,w}) \geq (d_0 - t_c) / d_0 = 2/3$ for the case with $t_c = 2$ and $d_0 = 6$. Considering that a larger β_{PE} will lead to a smaller α_{PE} , the optimal value for δ_2 should be given by the position where $\beta_{\text{PE}} = 2/3$. In this example, $\delta_2 = 1.75$.

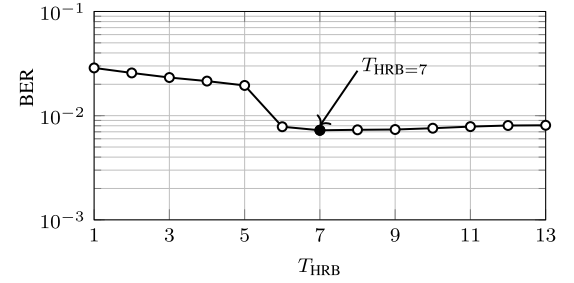
It needs to be mentioned that since the theoretical prediction does not consider the influence of the update of the marked bits (as iteration goes on), the marked thresholds obtained are slightly different from those obtained by numerical simulations. However, in the next section we will show that this mismatch will have a negligible impact on the performance of the proposed MLBM algorithm.

B. Other Parameters K_{MD} , T_{HRB} , f_{failure}

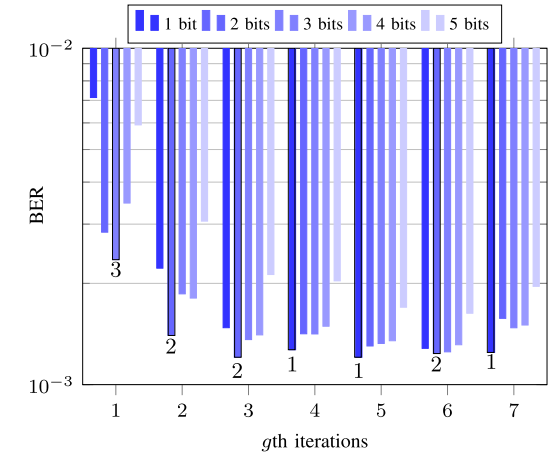
With the optimized marking thresholds $\delta_1 = 4.6$, $\delta_2 = 2$ and $\delta_3 = 0.87$, next we vary the values for K_{MD} . The results are shown in Fig. 6 (a). It shows that as the value of K_{MD} decreases, the BER performance will gradually become better, and when $K_{\text{MD}} = 2$ and $K_{\text{MD}} = 1$, we will have the lowest BER. To keep the complexity as low as possible, $K_{\text{MD}} = 2$ is used in the following simulations. In addition, we find that when $K_{\text{MD}} = 0$, indicating that all the decoding of the blocks in the window will performance MD, the BER performance will degrade significantly with high error floor. This is because the BDDs for the front blocks of the window are reliable enough (as there are few errors), however, are mistakenly



(a) K_{MD} : position threshold for performing MD



(b) T_{HRB} : threshold for unreliable HRB



(c) f_{failure} : the number of flipped bits at each iteration

Fig. 6. Optimization results of decoding parameters (a) K_{MD} , (b) T_{HRB} and (c) f_{failure} for the proposed decoding algorithm based on SCC with BCH(256,239,2) and 2-PAM. The obtained optimal values are highlighted in each sub-figure.

treated as miscorrections due to the inaccurate HRBs, and thus are rejected by the decoder.

Fig. 6 (b) shows the result for optimizing the unreliable HRB threshold T_{HRB} . We can see that $T_{\text{HRB}} = 7$ is the optimal value. When the value of T_{HRB} is too small, the BER performance will degrade, as some correct HRBs will be updated to RBs that will make several miscorrections unrecognized. On the other hand, too large T_{HRB} will lead to the inaccurate HRB (i.e., the HRB is in fact an error) have few chance to be removed from the HRB set, and thus cause performance degradation.

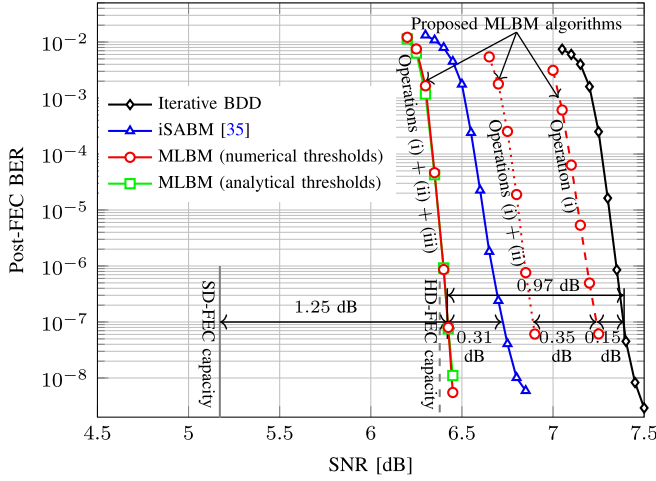


Fig. 7. BER performance of the proposed algorithm for SCC based on BCH (256,239,2) and 2-PAM. The three operations are (i) miscorrection detection, (ii) BF for the identified miscorrection followed by a second BDD trail, and (iii) BF for BDD failures followed by a second BDD trial. Iterative BDD and iSABM are used as the two baselines on HD decoding and weakly hybrid decoding, respectively.

Fig. 6 (c) shows the results for optimizing the number of flipped bits at each iteration for BDD failures. It shows that $\mathbf{f}_{\text{failure}} = [3, 2, 2, 1, 1, 2, 1]$ is the optimal one to have the lowest BER. Each element in $\mathbf{f}_{\text{failure}}$ is optimized one by one from 1 to 5, while the initial value is $\mathbf{f}_{\text{failure}} = [1, 1, 1, 1, 1, 1, 1]$. When one is optimized, the corresponding element in $\mathbf{f}_{\text{failure}}$ will be updated for the optimization of next element, while the elements that need to be optimized are still 1s. As expected, since there are more errors in the front of iterations, the decoder needs to flip more bits to make the errors left fall within the error-correcting capability of BDD.

V. PERFORMANCE ANALYSIS

With the optimized parameters, this section will analyze the BER performance of the proposed MLBM algorithm with 2-PAM and 8-PAM. At the same time, the marking accuracy will also be shown to validate the effectiveness of marked bit update algorithm.

A. BER Performance

Fig. 7 (red solid line) shows the BER performance of the proposed MLBM algorithm for SCC based on BCH (256,239,2), using marking thresholds obtained via numerical simulations. The modulation format is 2-PAM. The black and blue curves (with markers) show the performance of SCCs using iterative BDD and iSABM decoding [35], respectively, which are used as two baselines on HD decoding and weakly hybrid decoding, respectively. We can see that compared to conventional SCC using iterative BDD, the proposed algorithm can yield 0.97 dB gain at a BER of 10^{-7} . When compared to iSABM algorithm, the proposed algorithm has 0.31 dB gain. Moreover, benefiting from the MB update during the iterative decoding, the proposed algorithm shows a steeper waterfall region and lower error floor, when compared to iSABM and iterative BDD.

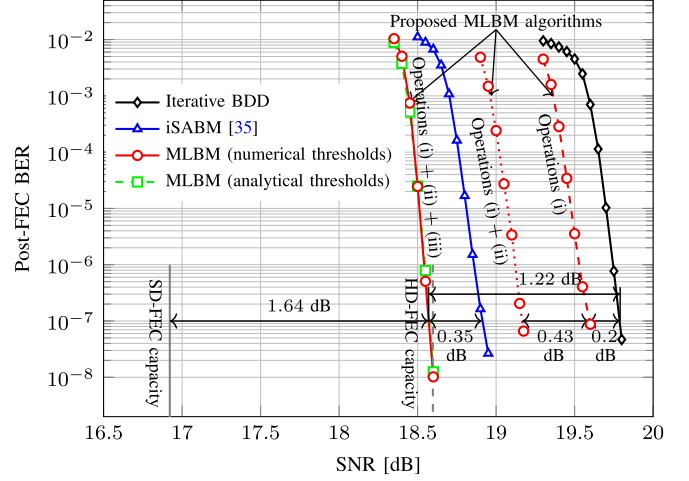


Fig. 8. BER performance of the proposed algorithm for SCC based on BCH (256,239,2) and 8-PAM.

From an information-theoretic point of view, the required SNR of a bit-interleaved coded modulation (BICM) systems in theory can be predicted by achievable information rates (AIRs) [40], [41]. In Fig. 7, the solid and dashed gray lines are the required SNR of HD- and SD-based 2-PAM BICM systems for 2-PAM predicted by AIRs. We can find that there is a very small gap (around 0.05 dB) between the required SNR in practice for the proposed algorithm and the predicted SNR via AIRs of HD-FEC, and the gap to the predicted SNR of SD-FEC is reduced to 1.25 dB.

As introduced in Sec. III, there are three key operations for the proposed algorithm: (i) miscorrection detection, (ii) BF for the identified BDD miscorrections followed by a second BDD trail, and (iii) BF for BDD failures followed by a second BDD trial. Therefore, to know about the contribution of the three operations, we also look into the performance of the proposed algorithm only with operation (i) (red dashed line), and operations (i) + (ii) (red dotted line). The red solid line represents the performance of the proposed MLBM algorithm with operations (i) + (ii) + (iii). We can see that compared to conventional SCC, MD for the proposed algorithm can bring 0.15 dB additional gain, while the combination of MD and BF for the identified BDD miscorrections can bring 0.35 dB additional gain. Further, performing BF for the BDD failures will greatly improve the achieved gain to 0.97 dB.

Moreover, the green line shown in Fig. 7 is the performance of the proposed MLBM algorithm using theoretically calculated optimal marking thresholds. It almost overlaps with the one using the optimized marking thresholds via numerical simulation. This demonstrates the effectiveness of the theoretical derivation for predicting optimal marking thresholds, which can save much time for parameter optimization for the practical use of the proposed algorithm.

Fig. 8 shows the BER performance of the proposed algorithm with 8-PAM. The red lines for MLBM use the same parameters with that for 2-PAM obtained via numerical simulations, while the green line for MLBM uses the theoretically predicted marking thresholds given in Appendix. The results

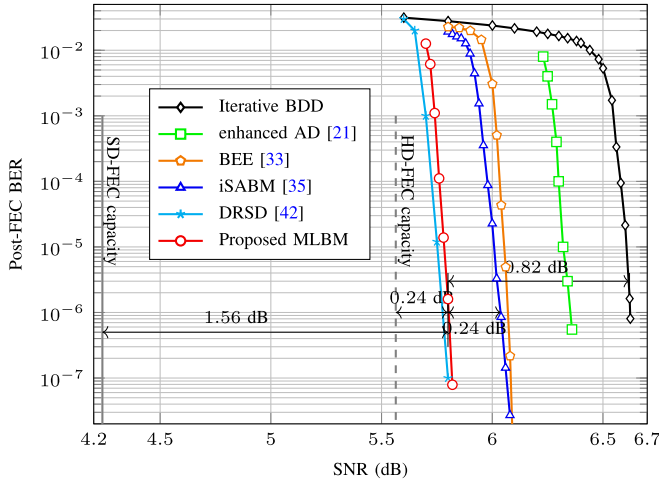


Fig. 9. Performance comparison of the proposed MLBM algorithm with other hybrid decoding algorithms. The SCC is based on BCH (254,230,3) and the modulation format is 2-PAM.

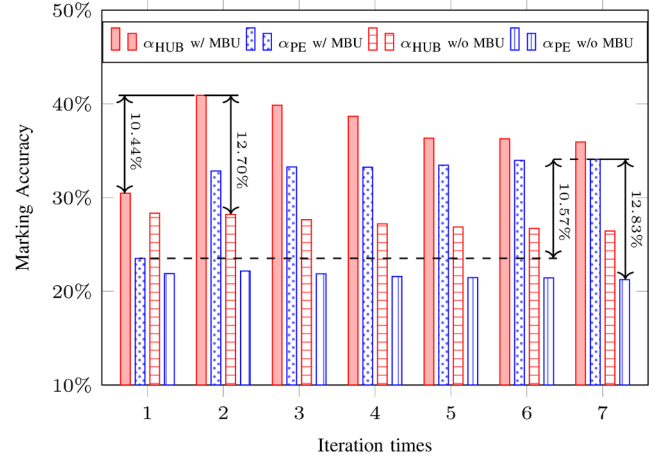
show that the achieved gains are slightly higher than that with 2-PAM, as interleaver is not used here.¹ When compared to conventional SCC using iterative BDD, the gain obtained by the proposed algorithm is 1.22 dB, while MD brings 0.2 dB gain and the combination of MD and BF for the identified miscorrections brings 0.43 dB gain, at a BER of 10^{-7} . With respect to iSABM, the achieved gain is 0.35 dB. Moreover, we have found that the required SNR in practice is slightly lower than that predicted by AIR of HD-FEC, while the gap to the predicted SNR of SD-FEC via AIR is 1.64 dB.

Further, we consider an SCC based on BCH (254,230,3) to make performance comparison with other hybrid decoding algorithms, including BEE [33], DRSD [42], and enhanced AD (an improved HD decoding algorithm) [21]. The BCH code is obtained by shortening one information bit from BCH (255,231,3). The resulted SCC code rate is $R = 0.811$. Other parameters, including $L = 7$ window size and $G = 10$ iterations, are the same as that in [33]. At the same time, a random interleaver is employed within each SCC block. The results are shown in Fig. 9 for 2-PAM. For the proposed algorithm, the optimized values for K_{MD} and T_{HRB} are 2 and 10, respectively. The number of flipped bits for BDD failure at each iteration is $\mathbf{f}_{failure} = [3, 2, 2, 1, 1, 2, 1, 1, 1, 1]$. It shows that the proposed MLBM algorithm has superior performance than BEE and enhanced AD, and exhibits similar performance to DRSD. In this case, the proposed algorithm outperforms iterative BDD and iSABM by 0.82 dB and 0.24 dB, respectively, and the gaps to the SNRs predicted by AIRs of HD-FEC and SD-FEC are 0.24 dB and 1.56 dB, respectively, at the BER of 10^{-6} .

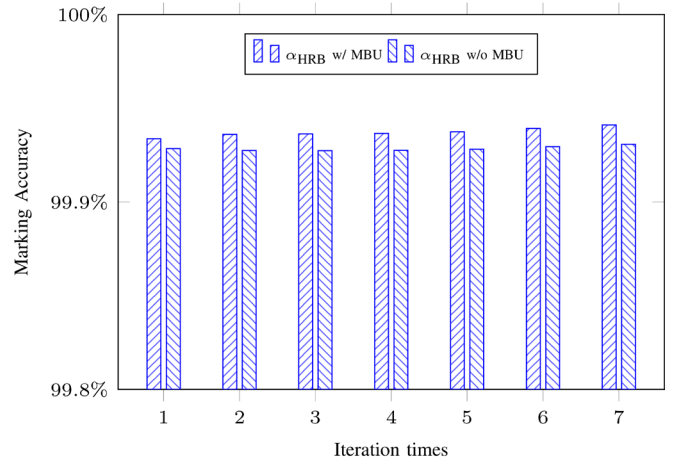
B. Marking Accuracy

To validate the effectiveness of the marked bit update, we also analyze the marking accuracy of HUBs, PEs and HRBs,

¹Due to the non-uniform distribution of the reliabilities in the large modulation size, the probability of burst errors becomes higher with respect to 2-PAM. As a result, miscorrections are more likely to happen, when an interleaver is not used.



(a) HUB/PE



(b) HRB

Fig. 10. Marking accuracies of HUBs and PEs (a) and HRBs (b) at each iteration for SCC based on BCH (256,239,2) and 2-PAM at SNR of 6.3 dB.

i.e., α_{HUB} , α_{PE} and α_{HRB} , at each iteration for the proposed algorithm with 2-PAM at SNR of 6.3 dB (corresponding to 1.65×10^{-3} BER shown in Fig. 7). The results are shown in Fig. 10, where the cases without MBU are also shown for comparison.

From Fig. 10 (a), we can see that without MBU, the marking accuracies of HUBs and PEs will gradually decrease as iteration goes on. This is because that as iteration goes on, more and more HUBs and PEs will be correctly decoded, which should be (but have not been) removed from the HUB or PE sets timely. When MBU is adopted, the marking accuracy of HUB can be maximum improved by 10.44% (from 30.46% to 40.9%) at the second iteration. Even though the values of α_{HUB} will gradually decrease after the second iteration, the marking accuracy is still significantly higher than that without MBU. For PEs, the marking accuracy α_{PE} will increase by 12.84% (from 21.24% to 34.08%) at the 7th iteration with respect to that with no MBU. When compared to the 1th iteration, the marking accuracy of PE will be improved by 10.57% (from 23.51% to 34.08%) from the 1th iteration to the 7th iteration.

Besides, from Fig. 10 (a) we can see that even though with the optimized marking thresholds and the MBU algorithm, we have $\alpha_{\text{HUB}} < 40\%$, let alone α_{PE} . This indicates that we only have less than 40% probability to correct errors via BF from the HUBs, which is not high obviously. Therefore, for the future work, other solutions, for example using side information provided by parity constraint violations [43], may need to be considered to increase α_{HUB} and α_{PE} to improve the efficiency of BF operation, and thus to yield more gains.

Fig. 10 (b) shows the marking accuracy of HRBs at each iteration. Generally, it shows that whether with MBU or not, the HRB marking accuracies are all beyond 99.9% with the optimized HRB threshold. Since the HRBs with no MBU are forbidden to be flipped or be decoded, the HRB marking accuracy keeps constant for all iterations. When compared to the case with no MBU, the case with MBU has higher HRB marking accuracy, even though the improvement is somewhat small.

VI. COMPLEXITY ANALYSIS

In addition to performance, the complexity of the algorithm is also of great concern to practical applications. In general, the complexity of the class of hybrid decoders is higher than that of HD decoders (because of the usage of soft information), but is lower than that of SD decoders (as the core is still HD decoding). How far to the HD decoders or SD decoders (in terms of the complexity) is mainly determined by how to use the soft information. In order to clarify the complexity of the proposed MLBM algorithm, this section will discuss the complexity increased caused by additional BDDs and increased memory usage. Note that in this section, all the analyses on complexity are for one decoding window including L SCC blocks.

A. Complexity Increase Caused by Additional BDDs

The complexity of the proposed algorithm mainly stems from the additional BDDs after BF. The relative complexity increase (RCI) caused by these additional BDDs is given by [34]

$$\eta_{\text{BDD}} \triangleq \frac{N_{\text{Proposed}} - N_{\text{Conv.}}}{N_{\text{Conv.}}}, \quad (9)$$

where N_{Proposed} and $N_{\text{Conv.}}$ in (9) are the total number of BDDs in the proposed decoding algorithm and the conventional decoding within a window, respectively.

Normally, BDD includes syndrome calculation, error estimation and error correction three steps for each input sequence. This is referred as full BDD (FBDD). However, when the calculated syndromes are zero, i.e., BDD thinks that there is no error in the input sequence, error estimation and error correction will not be performed. This will be referred as partial BDD (PBDD), which is less complex than full BDD. For SCCs at their working SNR region (where the post-FEC BER is significantly lower than the pre-FEC BER), partial BDD is required in most cases.

Fig. 11 (a) shows the number of FBDDs and PBDDs in the first and second BDD trials of a window at different SNRs. They are obtained by averaging the required number

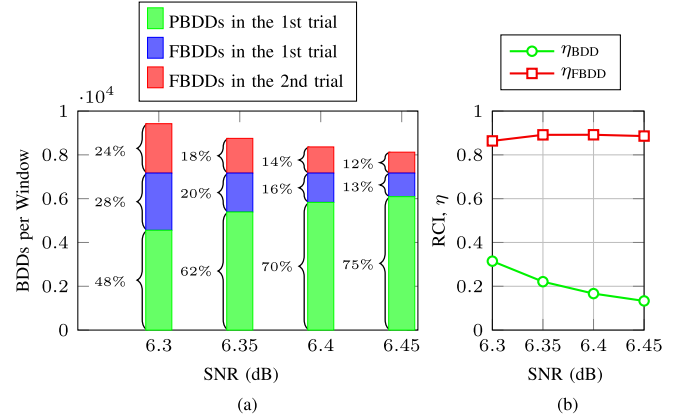


Fig. 11. The number of FBDDs and PBDDs in the first and second BDD trials (a) and the relative complexity increase caused by the additional BDDs (b) for SCC based on BCH (256,239,2) and 2-PAM.

of BDDs in 10,000 successive windows for SCC based on BCH (256,239,2) and 2-PAM. Let $N_{1,\text{FBDD}}$ and $N_{1,\text{PBDD}}$ be the number of FBDDs and PBDDs in the first BDD trial, respectively, the total number of BDDs for conventional SCCs is given by

$$N_{\text{Conv.}} = N_{1,\text{FBDD}} + N_{1,\text{PBDD}}. \quad (10)$$

Since the second BDD trial only deals with BDD miscorrections and failures, which indeed have errors in the sequence, the second BDD trials are always FBDDs with non-zero syndromes. Therefore, the total number of BDDs for the proposed algorithm is given by

$$N_{\text{Proposed}} = N_{1,\text{FBDD}} + N_{1,\text{PBDD}} + N_{2,\text{FBDD}}, \quad (11)$$

where $N_{2,\text{FBDD}}$ represent the number of FBDDs in the second BDD trail.

The green curve shown in Fig. 11 (b) is the calculated RCI resulted from the additional BDDs at different SNRs using equations (9)–(11). Generally, as SNR increases, the RCI η_{BDD} will decrease. When given the SNR of 6.3 dB, η_{BDD} is 31%. However, since the complexity of PBDD (with no error estimation and error correction) is far less than that of FBDD, the RCI obtained via (9)–(11) will be underestimated.

Further, we only consider the time-consuming FBDDs to estimated the RCI, i.e., $N'_{\text{Conv.}} = N_{1,\text{FBDD}}$ and $N'_{\text{Proposed}} = N_{1,\text{FBDD}} + N_{2,\text{FBDD}}$ in this case. By substituting $N'_{\text{Conv.}}$ and N'_{Proposed} into (9), we will have the upper bound of the RCI caused by the additional BDDs in the proposed algorithm. The red line in Fig. 11 (b) shows the results, indicating that the relative complexity increase almost keeps constant in this case, which are all beyond 80%. However, we can see from Fig. 11 (a) that due to the reduction of channel error probability, the proportion of full BDDs in the total BDDs will decrease significantly.

B. Increased Memory Usage

For the proposed algorithm, the required memories are mainly from four aspects:

- *HD-estimated bits*: There are L SCC blocks (each contain $W \times W$ bits) that need to be stored, i.e., LW^2 bits.

TABLE I
RELATIVE MEMORY INCREASE FOR THE PROPOSED ALGORITHM

Parameters	Value	Value
SCC code rate R	0.87	0.811
Block size $W \times W$	128×128	127×127
Window length L	9	7
Conflict threshold T_{HRB}	7	10
HRB conflict ratio γ	1.8%	1.9%
Required memory $\Gamma_{\text{Conv.}}$	147,456	112,903
Required memory Γ_{proposed}	490,050	379,354
Relative memory increase Γ	$\times 2.32$	$\times 2.36$

- *Marked bits*: As the marked bits includes four levels, including HRB, RB, UB and HUB, 2-bit marking is required for each HD-estimated bit. Therefore, the totally required memories are $2LW^2$ bits.
- *Decoding status matrix*: In the proposed algorithm, 2 bits are required to represent the four decoding statuses: decoding success with high confidence, decoding success, miscorrection, and decoding failure. Therefore, the totally required memories are $2(L+1)W$ bits to store the decoding statuses of a window.
- *Conflict counts for HRBs*: Even though many bits will be marked as HRBs (e.g., more than 80% for the considered SCC based on BCH (256,239,2) and 2-PAM), it is found that only small part of HRBs will conflict with BDD results. For example, for the proposed algorithm at SNR of 6.3 dB with 2-PAM, around 82.85% bits are marked as HRBs, while only 1.8% HRBs will conflict with the BDD results. To store the information on conflict counts for HRBs, an efficient way is to store the row index, column index, and its non-zero values. As the maximum value for the conflict count is T_{HRB} , it needs $\lceil \log_2 T_{\text{HRB}} \rceil$ -bit to represent these situations. Let γ be the ratio of the conflicting HRB counts to the total bits in an SCC block, the finally required memory to store the HRB conflict count can be obtained via $(2\lceil \log_2 W \rceil + \lceil \log_2 T_{\text{HRB}} \rceil)\gamma LW^2$.

As a result, the total required memory for the proposed algorithm can be written as

$$\Gamma_{\text{Proposed}} = LW^2 + 2LW^2 + 2(L+1)W + 2\lceil \log_2 W \rceil + \lceil \log_2 T_{\text{HRB}} \rceil \gamma LW^2. \quad (12)$$

And the relative memory increase is written as

$$\Gamma = (\Gamma_{\text{Proposed}} - \Gamma_{\text{Conv.}}) / \Gamma_{\text{Conv.}} = 2 + \frac{2(L+1)}{LW} + (2\lceil \log_2 W \rceil + \lceil \log_2 T_{\text{HRB}} \rceil) \gamma, \quad (13)$$

where $\Gamma_{\text{Conv.}}$ is the required memories for conventional SCC, which is given by $\Gamma_{\text{Conv.}} = LW^2$ for a window.

Table I shows the memory required for the two kinds of SCCs we have been analyzed in Sec. V. The parameters in detail are also shown in the table. According to Eq. (12) and Eq. (13), the relative memory increase of the proposed algorithm with the considered two kinds of SCC parameters are both around 2.3-fold with respect to conventional SCCs.

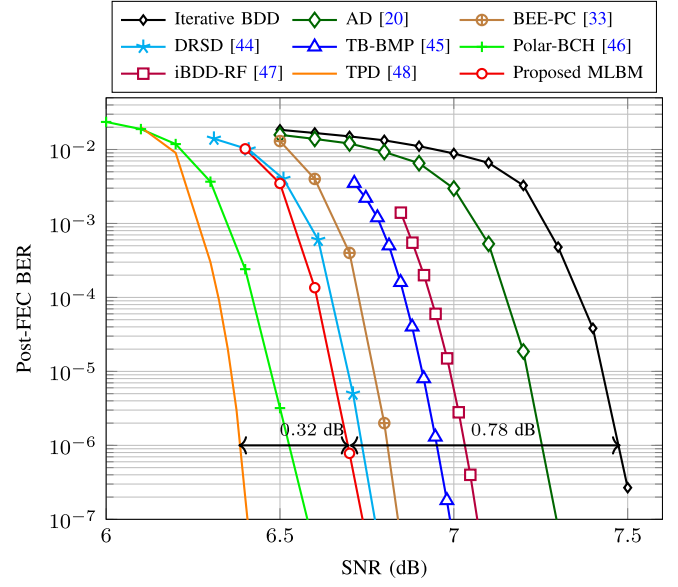


Fig. 12. Performance comparisons of different decoding algorithm for PCs with around 0.87 code rate and BPSK modulation format.

VII. EXTENSION TO PCs

Finally, we extend the proposed MLBM algorithm to PCs to verify its performance. Since row and column decodings in PCs are performed within a block, we perform MD and BF on all sequences within the block. This is unlike SCCs, where only part of the blocks in the decoding window perform MD. To facilitate the comparison of the differences between methods, BCH (256, 239, 2) is selected as the component code for PC, and an interleaver is considered in the system. The modulation format is BPSK. The maximum iteration times is $G = 10$. The threshold for unreliable HRB is $T_{\text{HRB}} = 6$ and the number of flipping bits for the decoding failures at each iteration is always 1. The marking thresholds are $\delta_1 = 4.5$, $\delta_2 = 1.75$ and $\delta_3 = 0.83$, derived from the theoretical prediction for 2-PAM case.

Fig. 12 shows the results of the proposed MLBM algorithm for PC. For comparison, two HD decoding algorithms (iterative BDD and AD [20]), five hybrid decoding algorithms (BEE-PC [33], DRSD [44], threshold-based binary message passing (TB-TMP) [45], Polar-BCH PC [46], and iBDD-RF [47]), and one soft-decision decoding algorithm (turbo product decoder (TPD) [48]) are also shown in the figure. In addition to Polar-BCH PC (which uses polar code with parameters of (256, 239) and BCH (256, 239, 2) as row and column component codes, respectively), these algorithms use BCH (255, 238, 2) or BCH (255, 239, 2) or BCH (256, 239, 2) as component code. The resulted PC code rates are all around 0.87. The results show that compared to iterative BDD, the proposed MLBM algorithm has 0.78 dB SNR gain. The performance achieved is significantly better than that of AD, BEE-PC, TB-BMP, and iBDD-RF algorithms, but has 0.32 dB performance loss when compared to the SD-based TPD case.

Particularly, the proposed MLBM algorithm presents slightly better performance than DRSD for PCs, while the two cases (MLBM and DRSD) share similar specific operations

such as detecting miscorrections via high reliable bits and updating soft information based on the decoding results. However, the two algorithms indeed have significant differences, mainly including three aspects:

- There is no sorting in MLBM, while DRSD needs to sort the bits according to their reliabilities to assign dynamic reliability scores and erase the most unreliable bits (this is not hardware-friendly).
- DRSD needs 5-bit memories to store the DRS (an integer in the range $[0, 31]$) for each coded bit. This means the relative memories increase is $\Gamma > 5$. However, as shown in Table I, the relative memories increase for MBLM is around 2.36.
- For each component decoding, MLBM requires the second BDD trial only when the first BDD is a miscorrection or a failure, while the majority cases only perform BDD once. By contrast, error-and-erasure decoding always runs BDD twice to decode the two test patterns. In this sense, the complexity of MLBM will be much lower than that of DRSD.

VIII. CONCLUSION AND FUTURE WORK

This paper proposes a hybrid hard-/soft-decision decoding algorithm for staircase codes to improve the performance. The core of the proposed algorithm is composed of four parts: bit marking, miscorrection detection, bit flipping and marked bit update. Based on channel soft information, the proposed algorithm initially marks bits into four reliability levels via three thresholds, which are then used to help the hard-decision component decoder to detect miscorrections and to decode those originally uncorrectable cases via bit flipping. Further, based on the decoding statuses and the marked bits, the algorithm proposes three kinds of priority levels for bit flipping and updates the marked bits timely. Numerical simulation results show that with limited complexity increase, this algorithm can bring up to 1.22 dB gains with respect to conventional SCC using hard-decision decoding, and can effectively lower the error floor as well. The results in this work demonstrate that hybrid decoding could be a good alternative for high-speed low-cost optical transmission systems offering substantial potential gains.

For future work, FPGA implementation of the proposed algorithm is an interesting topic to validate the performance and complexity analysis. This is important for the application of the proposed algorithm in practical optical fiber transmission systems.

APPENDIX THEORETICAL PREDICTION OF MARKING THRESHOLDS FOR 8-PAM

Using the well-known max-log approximation, the LLR λ_k for the k th ($k = 1, 2, 3$) bit in a 8-PAM symbol is given by

$$\lambda_k(y) \approx \frac{\gamma}{2} \left(\min_{i \in \mathcal{I}_{k,0}} |y - s_i|^2 - \min_{i \in \mathcal{I}_{k,1}} |y - s_i|^2 \right). \quad (14)$$

According to the bit-to-symbol Gray mapping rule shown in Fig. 13, the LLRs for the least significant bit (LSB), middle

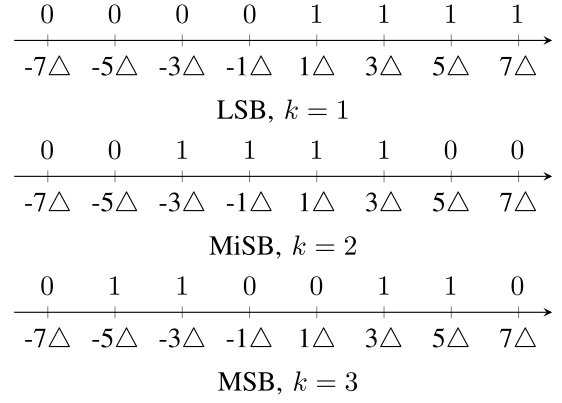


Fig. 13. Bit mapping for LSB, MiSB and MSB for 8-PAM with $\Delta = 1/\sqrt{21}$.

significant bit (MiSB), and most significant bit (MSB) can be rewritten as

$$\lambda_1(y) = \begin{cases} \frac{4}{\beta}y + \frac{12\Delta}{\beta}, & \text{if } y \leq -6\Delta \\ \frac{3}{\beta}y + \frac{6\Delta}{\beta}, & \text{if } -6\Delta \leq y \leq -4\Delta \\ \frac{2}{\beta}y + \frac{2\Delta}{\beta}, & \text{if } -4\Delta \leq y \leq -2\Delta \\ \frac{1}{\beta}y, & \text{if } -2\Delta \leq y \leq +2\Delta \\ \frac{2}{\beta}y - \frac{2\Delta}{\beta}, & \text{if } +2\Delta \leq y \leq +4\Delta \\ \frac{3}{\beta}y - \frac{6\Delta}{\beta}, & \text{if } +4\Delta \leq y \leq +6\Delta \\ \frac{4}{\beta}y - \frac{12\Delta}{\beta}, & \text{if } y \geq +6\Delta \end{cases} \quad (15)$$

and

$$\lambda_2(y) = \begin{cases} \frac{2}{\beta}y + \frac{10\Delta}{\beta}, & \text{if } y \leq -6\Delta \\ \frac{1}{\beta}y + \frac{4\Delta}{\beta}, & \text{if } -6\Delta \leq y \leq -2\Delta \\ \frac{2}{\beta}y + \frac{6\Delta}{\beta}, & \text{if } -2\Delta \leq y \leq 0 \\ -\frac{2}{\beta}y + \frac{6\Delta}{\beta}, & \text{if } 0 \leq y \leq +2\Delta \\ -\frac{1}{\beta}y + \frac{4\Delta}{\beta}, & \text{if } +2\Delta \leq y \leq +6\Delta \\ -\frac{2}{\beta}y + \frac{10\Delta}{\beta}, & \text{if } y \geq +6\Delta \end{cases} \quad (16)$$

and

$$\lambda_3(y) = \begin{cases} \frac{1}{\beta}y + \frac{6\Delta}{\beta}, & \text{if } y \leq -4\Delta \\ -\frac{1}{\beta}y - \frac{2\Delta}{\beta}, & \text{if } -4\Delta \leq y \leq 0 \\ \frac{1}{\beta}y - \frac{2\Delta}{\beta}, & \text{if } 0 \leq y \leq +4\Delta \\ -\frac{1}{\beta}y + \frac{6\Delta}{\beta}, & \text{if } y \geq +4\Delta \end{cases} \quad (17)$$

where $\Delta = 1/\sqrt{21}$.

According to the linear function for a Gaussian random variable $y \sim \mathcal{N}(x, 1/\gamma)$, the variable $ay + b$ can still be characterized by the Gaussian distribution with mean $ax + b$

$$p_2(\lambda|x) = \begin{cases} \frac{\alpha}{2} \sum_{u \in \{-1,1\}} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{2}+ux-5\Delta|^2}, & \text{if } \lambda \leq -\frac{2\Delta}{\beta} \\ \alpha \sum_{u \in \{-1,1\}} e^{-\frac{\gamma}{2}|\beta\lambda+ux-4\Delta|^2}, & \text{if } -\frac{2\Delta}{\beta} \leq \lambda \leq \frac{2\Delta}{\beta} \\ \frac{\alpha}{2} \sum_{u \in \{-1,1\}} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{2}+ux-3\Delta|^2}, & \text{if } \frac{2\Delta}{\beta} \leq \lambda \leq \frac{6\Delta}{\beta} \\ 0, & \text{if } \lambda \geq \frac{6\Delta}{\beta} \end{cases} \quad (19)$$

$$p_3(\lambda|x) = \begin{cases} \alpha \sum_{u \in \{-1,1\}} e^{-\frac{\gamma}{2}|\beta\lambda+ux-6\Delta|^2}, & \text{if } \lambda \leq -\frac{2\Delta}{\beta} \\ \alpha \sum_{u \in \{-1,1\}} \left[e^{-\frac{\gamma}{2}|\beta\lambda+ux-6\Delta|^2} + e^{-\frac{\gamma}{2}|\beta\lambda+ux+2\Delta|^2} \right], & \text{if } -\frac{2\Delta}{\beta} \leq \lambda \leq \frac{2\Delta}{\beta} \\ 0, & \text{if } \lambda \geq \frac{2\Delta}{\beta} \end{cases} \quad (20)$$

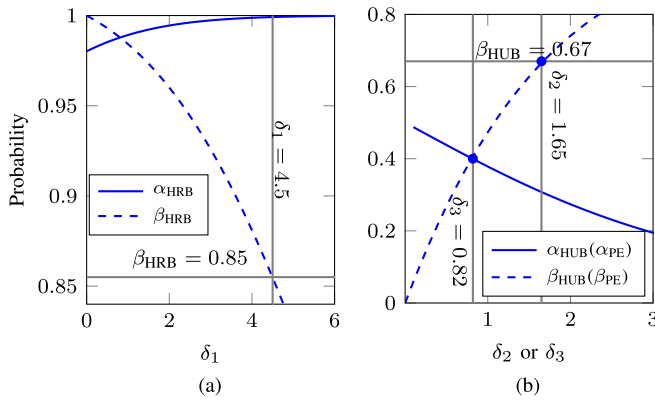


Fig. 14. The values of (a) α_{HRB} , β_{HRB} , (b) α_{PE} and β_{PE} , α_{HUB} and β_{HUB} against marking thresholds with 8-PAM at SNR of 18.45 dB.

and

$$\begin{cases} \alpha_{\text{PE}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_2}^0 p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_2}^{\delta_2} p_k(\lambda|x)} \\ \beta_{\text{PE}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_2}^0 p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\infty}^0 p_k(\lambda|x)} \end{cases} \quad (22)$$

and

$$\begin{cases} \alpha_{\text{HUB}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_3}^0 p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_3}^{\delta_3} p_k(\lambda|x)} \\ \beta_{\text{HUB}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\delta_3}^0 p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{-\infty}^0 p_k(\lambda|x)} \end{cases} \quad (23)$$

Fig. 14 shows the values of α_{HRB} , β_{HRB} , α_{PE} , β_{PE} , α_{HUB} and β_{HUB} against marking thresholds with 8-PAM at SNR of 18.45 dB. It shows that the optimal values for δ_1 , δ_2 and δ_3 should be around 4.5, 1.65 and 0.82, respectively, which are similar to the case for 2-PAM.

and variance a^2/γ . Therefore, the PDFs of the LLRs for 8-PAM can be given by Eqs. (18)–(20), as shown at the top of the page, which are

$$p_1(\lambda|x) = \begin{cases} \frac{\alpha}{4} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{4}-x-3\Delta|^2}, & \text{if } \lambda \leq -\frac{12\Delta}{\beta} \\ \frac{\alpha}{4} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{3}-x-2\Delta|^2}, & \text{if } -\frac{12\Delta}{\beta} \leq \lambda \leq -\frac{6\Delta}{\beta} \\ \frac{\alpha}{2} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{2}-x-\Delta|^2}, & \text{if } -\frac{6\Delta}{\beta} \leq \lambda \leq -\frac{2\Delta}{\beta} \\ \alpha e^{-\frac{\gamma}{2}|\beta\lambda-x|^2}, & \text{if } -\frac{2\Delta}{\beta} \leq \lambda \leq \frac{2\Delta}{\beta} \\ \frac{\alpha}{2} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{2}-x+\Delta|^2}, & \text{if } \frac{2\Delta}{\beta} \leq \lambda \leq \frac{6\Delta}{\beta} \\ \frac{\alpha}{2} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{3}-x+2\Delta|^2}, & \text{if } \frac{6\Delta}{\beta} \leq \lambda \leq \frac{12\Delta}{\beta} \\ \frac{\alpha}{4} e^{-\frac{\gamma}{2}|\frac{\beta\lambda}{4}-x+3\Delta|^2}, & \text{if } \lambda \geq \frac{12\Delta}{\beta} \end{cases} \quad (18)$$

Then the expressions for α_{HRB} , β_{HRB} , α_{PE} , β_{PE} , α_{HUB} and β_{HUB} for 8-PAM can be written as

$$\begin{cases} \alpha_{\text{HRB}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{\delta_1}^{\infty} p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \left[\int_{-\delta_1}^{\delta_1} p_k(\lambda|x) + \int_{\delta_1}^{\infty} p_k(\lambda|x) \right]} \\ \beta_{\text{HRB}} = \frac{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_{\delta_1}^{\infty} p_k(\lambda|x)}{\sum_k \frac{1}{3} \sum_x \frac{1}{4} \int_0^{\infty} p_k(\lambda|x)} \end{cases} \quad (21)$$

REFERENCES

- [1] D. Zhang et al., “Technological prospection and requirements of 800G transmission systems for ultra-long-haul all-optical terrestrial backbone networks,” *J. Lightw. Technol.*, vol. 41, no. 12, pp. 3774–3782, Jun. 15, 2023.
- [2] W. Wang et al., “Real-time FPGA investigation of potential FEC schemes for 800G-ZR/ZR+ forward error correction,” *J. Lightw. Technol.*, vol. 41, no. 3, pp. 926–933, Feb. 1, 2023.
- [3] K. Tao et al., “Low-complexity forward error correction for 800G unamplified campus link,” in *Proc. 20th Int. Conf. Opt. Commun. Netw. (ICOCN)*, Shenzhen, China, Aug. 2022, pp. 1–3.
- [4] W. Wang et al., “Investigation of potential FEC schemes for 800G-ZR forward error correction,” in *Proc. Opt. Fiber Commun. Conf. Exhib. (OFC)*, San Diego, CA, USA, Mar. 2022, pp. 1–3.
- [5] A. G. I. Amat, G. Liva, and F. Steiner, “Coding for optical communications—Can we approach the Shannon limit with low complexity?” in *Proc. 45th Eur. Conf. Opt. Commun. (ECOC)*, Dublin, Ireland, Sep. 2019, pp. 1–4.
- [6] G. Tzimpragos, C. Kachris, I. B. Djordjevic, M. Cvijetic, D. Soudris, and I. Tomkos, “A survey on FEC codes for 100G and beyond optical networks,” *IEEE Commun. Surveys Tuts.*, vol. 18, no. 1, pp. 209–221, 1st Quart., 2016.
- [7] A. Zokaie, D. Truhachev, and K. El-Sankary, “Memory optimized hardware implementation of open FEC encoder,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 10, pp. 1548–1552, Oct. 2022.
- [8] P. Elias, “Error-free coding,” *Trans. IRE Prof. Group Inf. Theory*, vol. 4, no. 4, pp. 29–37, Sep. 1954.
- [9] B. P. Smith, A. Farhood, A. Hunt, F. R. Kschischang, and J. Lodge, “Staircase codes: FEC for 100 Gb/s OTN,” *J. Lightw. Technol.*, vol. 30, no. 1, pp. 110–117, Jan. 1, 2012.

- [10] A. Y. Sukmadji, U. Martínez-Peñas, and F. R. Kschischang, "Zipper codes," *J. Lightw. Technol.*, vol. 40, no. 19, pp. 6397–6407, Oct. 1, 2022.
- [11] ITU, *OTU4*, document ITU-T Recommendation G.709.2/Y.1331.2, Jul. 2018.
- [12] *Implement. Agreement 400ZR*, Optical Internetworking Forum, Fremont, CA, USA, Mar. 2020.
- [13] Z. Liu, B. Feng, J. Jiao, K. Liang, S. Wu, and Q. Zhang, "Performance analysis of soft decoding algorithms for polar-staircase coding scheme," in *Proc. 10th Int. Conf. Wireless Commun. Signal Process. (WCSP)*, Hangzhou, China, Oct. 2018, pp. 1–6.
- [14] C. Condo, V. Bioglio, and I. Land, "Staircase construction with non-systematic polar codes," in *Proc. Opt. Fiber Commun. Conf. Exhib. (OFC)*, San Diego, California, Mar. 2020, pp. 1–3.
- [15] V. B. Wijekoon, E. Viterbo, and Y. Hong, "LDPC-staircase codes for soft decision decoding," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, May 2020, pp. 1–6.
- [16] O. Griebel, M. Herrmann, B. Hammoud, and N. Wehn, "Energy-efficient cancellation list flip decoder of polar codes with fast decoding of special nodes," *J. Commun. Netw.*, vol. 23, no. 6, pp. 417–432, Dec. 2021.
- [17] L. Sassatelli, S. K. Chilappagari, B. Vasic, and D. Declercq, "Two-bit message passing decoders for LDPC codes over the binary symmetric channel," in *Proc. IEEE Int. Symp. Inf. Theory*, Jun. 2009, pp. 2156–2160.
- [18] F. Ivanov, V. Morishnik, and E. Krouk, "Improved generalized successive cancellation list flip decoder of polar codes with fast decoding of special nodes," *J. Commun. Netw.*, vol. 23, no. 6, pp. 417–432, Dec. 2021.
- [19] L. Holzbaur, H. Bartz, and A. Wachter-Zeh, "Improved decoding and error floor analysis of staircase codes," *Designs, Codes Cryptography*, vol. 87, nos. 2–3, pp. 647–664, Nov. 2018.
- [20] C. Häger and H. D. Pfister, "Approaching miscorrection-free performance of product codes with anchor decoding," *IEEE Trans. Commun.*, vol. 66, no. 7, pp. 2797–2808, Jul. 2018.
- [21] X. Zhao, S. Zhao, and Z. Li, "Enhanced anchor decoder for staircase codes with hard reliability scores," *IEEE Commun. Lett.*, vol. 26, no. 12, pp. 2826–2830, Dec. 2022.
- [22] M. Shehadeh, F. R. Kschischang, and A. Y. Sukmadji, "Generalized staircase codes with arbitrary bit degree," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, San Diego, CA, USA, Mar. 2024, p. W4C.2.
- [23] M. Barakatain and F. R. Kschischang, "Low-complexity concatenated LDPC-staircase codes," *J. Lightw. Technol.*, vol. 36, no. 12, pp. 2443–2449, Jun. 15, 2018.
- [24] Y. Cai et al., "FPGA investigation on error-flare performance of a concatenated staircase and Hamming FEC code for 400G inter-data center interconnect," *J. Lightw. Technol.*, vol. 37, no. 1, pp. 188–195, Jan. 1, 2019.
- [25] T. Mehmood, M. P. Yankov, S. Iqbal, and S. Forchhammer, "Flexible multilevel coding with concatenated polar-staircase codes for M-QAM," *IEEE Trans. Commun.*, vol. 69, no. 2, pp. 728–739, Feb. 2021.
- [26] T. Wettlin, S. Calabró, N. Stojanović, Y. Lin, and T. Rahman, "Investigation of concatenated KP4 FEC with single-parity-check codes for short-reach IM/DD systems," in *Proc. Opt. Fiber Commun. Conf. (OFC)*, San Diego, CA, USA, Mar. 2024, p. M2B.5.
- [27] M. Barakatain, Y. Hashemi, B. Karimi, H. Ebrahimzad, and C. Li, "Low-complexity zipper-LDPC and low-latency zipper-BCH concatenated codes," *J. Lightw. Technol.*, vol. 41, no. 24, pp. 7328–7335, Dec. 15, 2023.
- [28] M. Zhu, M. Jiang, C. Zhao, and F. Yu, "A low-complexity RS-SPC product coding scheme for optical systems," *IEEE Trans. Commun.*, vol. 71, no. 2, pp. 658–673, Feb. 2023.
- [29] M. Zhu, M. Jiang, and C. Zhao, "A class of staircase codes with mixed components and its low-complexity decoding," *IEEE Trans. Commun.*, vol. 72, no. 4, pp. 1895–1911, Apr. 2023.
- [30] S. Miao et al., "Trends in channel coding for 6G," *Proc. IEEE*, vol. 112, no. 7, pp. 653–675, Jun. 2024.
- [31] A. Sheikh, A. G. i. Amat, and G. Liva, "Binary message passing decoding of product-like codes," *IEEE Trans. Commun.*, vol. 67, no. 12, pp. 8167–8178, Dec. 2019.
- [32] A. Sheikh, A. G. i. Amat, and G. Liva, "Binary message passing decoding of product codes based on generalized minimum distance decoding: (Invited Paper)," in *Proc. 53rd Annu. Conf. Inf. Sci. Syst. (CISS)*, Baltimore, MD, USA, Mar. 2019, pp. 1–5.
- [33] A. Sheikh, A. G. i. Amat, and A. Alvarado, "Novel high-throughput decoding algorithms for product and staircase codes based on error-erasure decoding," *J. Lightw. Technol.*, vol. 39, no. 15, pp. 4909–4922, Aug. 1, 2021.
- [34] Y. Lei et al., "Improved decoding of staircase codes: The soft-aided bitmarking (SABM) algorithm," *IEEE Trans. Commun.*, vol. 67, no. 12, pp. 8220–8232, Dec. 2019.
- [35] Y. Lei, B. Chen, G. Liga, A. Balatsoukas-Stimming, K. Sun, and A. Alvarado, "A soft-aided staircase decoder using three-level channel reliabilities," *J. Lightw. Technol.*, vol. 39, no. 19, pp. 6191–6203, Oct. 1, 2021.
- [36] S. Miao, L. Rapp, and L. Schmalen, "Improved soft-aided decoding of product codes with dynamic reliability scores," *J. Lightw. Technol.*, vol. 40, no. 22, pp. 7279–7288, Nov. 15, 2022.
- [37] Y. Shen, Y. Ren, and A. Burg, "Iterative ordered statistics decoding of product codes," in *Proc. 12th Int. Symp. Topics Coding (ISTC)*, Brest, France, Sep. 2023, pp. 1–5.
- [38] O. M. Govardhan, "Fundamentals and classification of hybrid electric vehicles," *Int. J. Eng. Techn.*, vol. 3, no. 5, pp. 194–198, Oct. 2017.
- [39] G. Forney, "Generalized minimum distance decoding," *IEEE Trans. Inf. Theory*, vol. IT-12, no. 2, pp. 125–131, Apr. 1966.
- [40] G. Caire, G. Taricco, and E. Biglieri, "Bit-interleaved coded modulation," *IEEE Trans. Inf. Theory*, vol. 44, no. 3, pp. 927–946, Mar. 1998.
- [41] A. Alvarado, T. Fehenberger, B. Chen, and F. M. J. Willems, "Achievable information rates for fiber optics: Applications and computations," *J. Lightw. Technol.*, vol. 36, no. 2, pp. 424–439, Jan. 15, 2018.
- [42] L. Rapp, S. Miao, and L. Schmalen, "Optimized soft-aided decoding of ofec and staircase codes," in *Proc. ECOC*, 2024, pp. 539–542.
- [43] V. Y. Krachkovsky, J. J. Ashley, C. J. Williamson, and G. Feyh, "Syndrome Reed–Solomon decoding using bit reliabilities," *IEEE Trans. Magn.*, vol. 44, no. 1, pp. 223–227, Jan. 2008.
- [44] S. Miao, L. Rapp, and L. Schmalen, "Improved soft-aided error-erasure decoding of product codes with dynamic reliability scores," in *Proc. Opt. Fiber Commun. Conf. Exhib. (OFC)*, San Diego, CA, USA, Mar. 2022, pp. 1–3.
- [45] X. Zhao, S. Zhao, Q. Deng, Z. Li, and X. Tang, "A threshold-based binary message passing decoder with memory for product codes," *IEEE Trans. Commun.*, vol. 72, no. 9, pp. 5241–5254, Sep. 2024.
- [46] B. Chen, W. Yin, Y. Lei, Q. Huang, and X. Li, "Hybrid decoding of asymmetric product codes with systematic polar and BCH codes," *IEEE Commun. Lett.*, vol. 28, no. 10, pp. 2223–2227, Oct. 2024.
- [47] G. Li, S. Wang, and S. Zhao, "Iterative bounded distance decoding with random flipping for product-like codes," *IEEE Trans. Commun.*, vol. 73, no. 5, pp. 2864–2875, May 2025.
- [48] R. M. Pyndiah, "Near-optimum decoding of product codes: Block turbo codes," *IEEE Trans. Commun.*, vol. 46, no. 8, pp. 1003–1010, Aug. 1998.